

Index

Sr No	Date	Practical	Page No	Sign
1		a) Encrypting and Decrypting Data Using a Hacker Tool b) Encrypting and Decrypting Data Using OpenSSL c) Hashing a Text File with OpenSSL and Verifying Hashes	2	
2		a) Examining Telnet and SSH in Wireshark b) Investigating an Attack on a Windows Host c) Investigating a Malware Exploit	8	
3		a) Demonstrate the use of Snort and Firewall Rules b) Demonstrate Extract an Executable from a PCAP c) Demonstrate a practical for Exploring DNS Traffic	19	
4		a) Using Wireshark to Examine HTTP and HTTPS Traffic b) Exploring Processes, Threads, Handles, and Windows Registry	26	
5		Perform a practical to Attack on a MySQL Database by using PCAP file.	35	
6		Create your own syslog Server	38	
7		Configure your Linux system to send syslog messages to a syslog server and Read them	43	
8		Install and Run Splunk on Linux	52	

Practical No 1

Part A: Encrypting and Decrypting Data Using a Hacker Tool

ZIP files are commonly used for compressing and archiving multiple files. To enhance security, ZIP files can be protected using passwords, encrypting the contents. However, weak passwords can be cracked using brute-force and dictionary attacks.

John the Ripper (JtR) is a powerful password-cracking tool in Kali Linux that uses different attack strategies such as brute force and dictionary attacks to recover passwords from encrypted files.

In this practical, we:

Encrypt a ZIP file with a password.

Extract the password hash from the ZIP file using zip2john.

Use John the Ripper to crack the password.

Commands Used

Step 1: Creating an Encrypted ZIP File

To create a password-protected ZIP file, use the following command:

```
zip -e abcd1.zip smile.txt
```

-e enables encryption.

abcd1.zip is the name of the output ZIP file.

smile.txt is the file to be compressed and encrypted.

After running the command, you will be prompted to enter and verify a password. Here, the password used was "a".

Step 2: Extracting the Hash from the ZIP File

Before cracking the password, we need to extract the hash using the zip2john tool:

```
zip2john abcd1.zip > hash.txt
```

zip2john extracts the hash from the encrypted ZIP file.

> hash.txt saves the extracted hash into a file named hash.txt.

Step 3: Cracking the Password Using John the Ripper

Once we have the hash, we use John the Ripper to crack the password:

```
john hash.txt
```

John the Ripper automatically starts attacking the hash using its default wordlist.

If the password is weak, John will successfully recover it in a short time.

Once cracked, John displays the recovered password on the screen.

Command to encrypt using hash and key:

```
File Actions Edit View Help

(kali@kali)-[~]
$ echo This is a secure file>smile.txt

(kali@kali)-[~]
$ dir
0      Documents  key      Pictures  smile.txt  Videos
Desktop Downloads  Music    Public    Templates

(kali@kali)-[~]
$ zip -e abcd1.zip smile.txt
Enter password:
Verify password:
adding: smile.txt (stored 0%)
```

Command to crack the password using hacker tool John the Ripper

```
(kali@kali)-[~]
$ zip2john abcd1.zip>hash.txt
ver 1.0 efh 5455 efh 7875 abcd1.zip/smile.txt PKZIP Encr: 2b chk, TS_chk, cmp
len=34, decmplen=22, crc=6532B9A1 ts=9F44 cs=9f44 type=0

(kali@kali)-[~]
$ john hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
a          (abcd1.zip/smile.txt)
1g 0:00:00:00 DONE 2/3 (2024-10-21 20:02) 10.00g/s 438130p/s 438130c/s 438130
C/s 123456.. Peter
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Conclusion

This practical demonstrates how password-protected ZIP files can be cracked using John the Ripper if weak passwords are used. It highlights the need for strong passwords (longer than 12 characters, mixed with uppercase, lowercase, numbers, and special characters) to enhance security.

Cybersecurity professionals and ethical hackers use such techniques to assess system vulnerabilities and educate users on better security practices. However, unauthorized cracking of encrypted files is illegal and should only be performed for ethical and educational purposes.

Part B: Encrypting and Decrypting Data Using OpenSSL

Encryption is a crucial process in cybersecurity that ensures data confidentiality by converting plaintext into an unreadable format, known as ciphertext. Decryption reverses the process, converting ciphertext back to its original form. OpenSSL is an open-source cryptographic library that provides various encryption and decryption functionalities using different algorithms.

In this practical, we explore symmetric encryption, where the same key is used for both encryption and decryption. Two widely used symmetric encryption algorithms are:

Triple Data Encryption Standard (3DES - des-ede3-cbc)

An enhancement of the original DES algorithm that applies encryption three times for increased security.

Uses a 168-bit key (three 56-bit keys) in CBC (Cipher Block Chaining) mode.

Advanced Encryption Standard (AES)

A modern encryption standard with key sizes of 128, 192, or 256 bits.

Faster and more secure than 3DES, widely used in modern cryptographic applications.

Commands Used

Step 1: Generating a Random Key

To perform encryption, a cryptographic key is required. We generate a 64-byte key using the following command:

```
openssl rand -hex -out key 64
```

This creates a random 64-byte hexadecimal key and stores it in a file named key.

Step 2: Creating and Encrypting a File (3DES Algorithm)

Create a sample text file (e.g., smile.txt) containing some data.

Encrypt the file using the Triple DES (3DES) algorithm with CBC mode:

```
openssl des-ede3-cbc -in smile.txt -out zmile -e -a
```

des-ede3-cbc specifies the Triple DES encryption with CBC mode.

-in smile.txt specifies the input file to be encrypted.

-out zmile defines the output encrypted file.

-e stands for encryption.

-a enables Base64 encoding for easy readability.

Step 3: Decrypting the Encrypted File

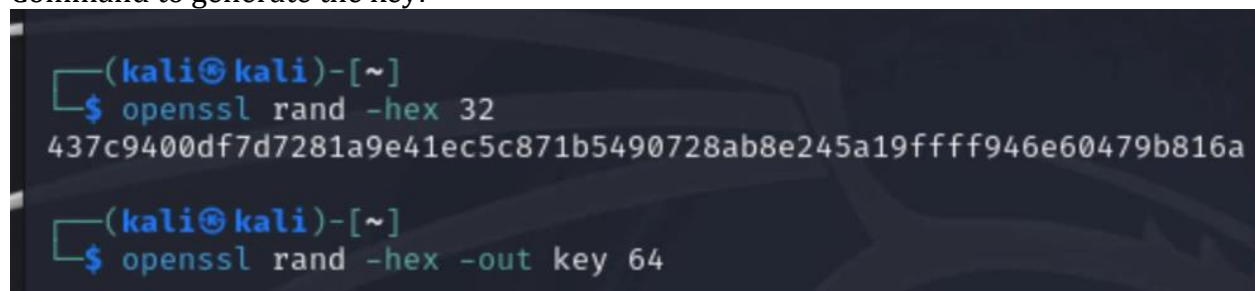
To retrieve the original content from the encrypted file, use the decryption command:

```
openssl des-ede3-cbc -in zmile -out abcd -d -a
```

-d stands for decryption.

The decrypted file will be stored as abcd.

Command to generate the key:



```
(kali㉿kali)-[~]  
$ openssl rand -hex 32  
437c9400df7d7281a9e41ec5c871b5490728ab8e245a19ffff946e60479b816a  
  
(kali㉿kali)-[~]  
$ openssl rand -hex -out key 64
```

Command to encrypt the file:

```
➔ cat key
994107c3d6f6185a32ae71606d8555e95a2631ac1c4c2224a73cd469a26973774298f920da42f
b76357d30d87049290f9e7cc0e0b003adf7e456bcb1d7cfa611

(kali㉿kali)-[~]
$ echo This is a confidential file > smile.txt

(kali㉿kali)-[~]
$ dir
0      Documents  key      Pictures  smile.txt  Videos
Desktop Downloads  Music    Public    Templates

(kali㉿kali)-[~]
$ openssl des-ede3-cbc -in smile.txt -out zmile -e -a
enter DES-EDE3-CBC encryption password:
Verifying - enter DES-EDE3-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Command to decrypt the file:

```
(kali㉿kali)-[~]
$ openssl des-ede3-cbc -in zmile -out abcd -d -a
enter DES-EDE3-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(kali㉿kali)-[~]
$ dir
0      Desktop  Downloads  Music    Public  Templates  zmile
abcd  Documents  key      Pictures  smile.txt  Videos

(kali㉿kali)-[~]
$ cat abcd
This is a confidential file
```

Conclusion

This practical demonstrates the process of encrypting and decrypting files using OpenSSL in Kali Linux. The 3DES and AES algorithms provide different levels of security, with AES being the preferred choice for modern encryption due to its efficiency and stronger security. OpenSSL remains an essential tool for implementing cryptographic techniques in cybersecurity.

Part C: Hashing a Text File with OpenSSL and Verifying Hashes

A hash function is a mathematical algorithm that converts input data into a fixed-size unique string known as a hash or digest. Hashing is widely used in cybersecurity for verifying data integrity, password storage, and digital signatures.

SHA-256 (Secure Hash Algorithm 256-bit)

SHA-256 is a widely used cryptographic hash function that produces a 256-bit hash value (64 hexadecimal characters). It is part of the SHA-2 family and is used in applications like digital signatures, blockchain technology, and secure password storage.

Key Properties of Cryptographic Hash Functions:

Deterministic – The same input always produces the same hash.

Fast Computation – Hashes are generated quickly.

Irreversibility – It is computationally infeasible to retrieve the original input from the hash.

Collision Resistance – Two different inputs should not produce the same hash.

Avalanche Effect – A small change in the input causes a significant change in the output hash.

Commands Used

Step 1: Hashing a Text File Using SHA-256

To generate a hash for a text file (smile.txt), use the following command:

```
openssl sha256 -hex -out abcd.sha smile.txt
```

sha256 specifies the SHA-256 hashing algorithm.

-hex ensures the hash is displayed in hexadecimal format.

-out abcd.sha saves the output hash in the file abcd.sha.

smile.txt is the input file whose hash is generated.

Step 2: Hashing Another Text File with a Slight Change

Create another file (smile1.txt) with just a single letter change and generate its hash:

```
openssl sha256 -hex -out abcd1.sha smile1.txt
```

The only difference between smile.txt and smile1.txt is a minor change in a single alphabet. However, when comparing abcd.sha and abcd1.sha, the hash values are drastically different due to the avalanche effect.

Step 3: Verifying the Hash Values

To check the generated hashes, use:

```
cat abcd.sha
```

```
cat abcd1.sha
```

This will display the SHA-256 hash values for both files, showing how a small change in the input leads to a completely different hash output.

The practical clearly demonstrates the avalanche effect, where even a single character change results in a drastically different hash. This is crucial for security because even minor modifications in a file will be detectable, ensuring data integrity and preventing tampering.

Command to create file and their hashes

```
(kali㉿kali)-[~]
└─$ echo Welcome to ismile academy> smile.txt

(kali㉿kali)-[~]
└─$ echo Welcome to esmile academy> smile1.txt

(kali㉿kali)-[~]
└─$ dir
0      Documents  Music      Public      smile.txt  Videos
Desktop Downloads  Pictures   smile1.txt Templates

(kali㉿kali)-[~]
└─$ openssl sha256 -hex -out abcd.sha smile.txt

(kali㉿kali)-[~]
└─$ openssl sha256 -hex -out abcd1.sha smile1.txt
```

Command to check the hashes of the two files

```
(kali㉿kali)-[~]
└─$ cat abcd.sha
SHA2-256(smile.txt)= ec5a8a902c8e92ba285df8e13ec98d3ce95804e2b86557d30081286a
5c1e2b8c

(kali㉿kali)-[~]
└─$ cat abcd1.sha
SHA2-256(smile1.txt)= 72e40c3648bb7848194d93fbc865597805830902258b79077097711
cb4d6d4c3
```

Conclusion

This experiment highlights the importance of cryptographic hash functions in data security and integrity verification. It shows how hashing ensures data consistency and how minor changes in input can significantly alter the output hash. SHA-256 is widely used for password hashing, digital signatures, and blockchain technology, making it a fundamental tool in cybersecurity.

Practical No 2

Part A: Examining Telnet and SSH in Wireshark

The objective of this practical is to analyze the security differences between Telnet and SSH using Wireshark by capturing network packets. This experiment demonstrates how Telnet transmits credentials in plaintext, making it insecure, whereas SSH provides encrypted communication.

Introduction

Telnet and Its Security Concerns

Telnet (Telecommunication Network) is a protocol used for **remote command-line access to a system over a network**. However, Telnet **lacks encryption**, meaning that all transmitted data, including login credentials and commands, is sent in **plaintext**. This makes Telnet highly vulnerable to man-in-the-middle (**MITM**) attacks, where an attacker can easily intercept and read network traffic.

Secure Shell (SSH) – A Secure Alternative

SSH (Secure Shell) is a secure **replacement** for Telnet. SSH uses **encryption and authentication** mechanisms to protect data, ensuring that even if network traffic is intercepted, it remains unreadable to attackers.

Commands Used

Step 1: Installing and Setting Up a Telnet Server

To set up a Telnet server on Kali Linux, install the required tools:

```
sudo apt install busybox-static -y
```

```
sudo apt install telnet -y
```

busybox-static provides a **lightweight Telnet server**.

telnet installs the **Telnet client** for connecting to the server.

Step 2: Starting the Telnet Server

Run the following command to start a Telnet server on port 23 (default Telnet port):

```
sudo busybox telnetd -f -p 23
```

telnetd starts the **Telnet daemon**.

-f runs it in the **foreground**.

-p 23 specifies the **port number** (23 is the default for Telnet).

Step 3: Capturing Telnet Traffic in Wireshark

Open Wireshark.

Select the network interface (e.g., lo for localhost or the active network interface).

Start packet capture.

Use the following command to connect to the Telnet server:

```
telnet localhost
```

Enter the username and password when prompted.

Step 4: Analyzing Telnet Traffic

In Wireshark, filter packets using:
telnet

Observe that the username and password are visible in plaintext, proving that Telnet is insecure.

Observation: Why Telnet is Insecure

All data, including credentials, is transmitted in unencrypted form.

An attacker sniffing the network traffic can easily capture usernames, passwords, and commands.

This makes Telnet highly vulnerable to eavesdropping, credential theft, and session hijacking.

Conclusion

This practical demonstrates the security risks of using Telnet for remote access. Since Telnet transmits data in plaintext, it should never be used on public or untrusted networks. Instead, SSH (Secure Shell) should be used, as it encrypts all communication, protecting credentials and commands from being intercepted.

Part B: Investigating an Attack on a Windows Host

Objective:

The objective of this practical is to investigate a security incident on a Windows host using Wireshark to analyze network traffic. This experiment demonstrates how to identify malicious activities by capturing and examining packets, providing insights into potential security breaches.

Introduction

In cybersecurity, monitoring network traffic is crucial for detecting unauthorized activities. Attackers often exploit vulnerabilities in Windows hosts to gain unauthorized access or deploy malware. By analyzing network packets with tools like Wireshark, security professionals can uncover evidence of such attacks, including suspicious connections, data exfiltration, or command-and-control communications.

Commands and Steps

Step 1: Setting Up the Environment

1. Install Wireshark:
 - Download and install Wireshark from the [official website](#).
2. Prepare the Windows Host:
 - Ensure the Windows machine is connected to the network and has administrative privileges.

Step 2: Capturing Network Traffic

1. Start Wireshark:
 - Launch Wireshark on the Windows host.
2. Select the Network Interface:
 - Choose the appropriate network interface (e.g., Ethernet or Wi-Fi) to monitor.
3. Begin Packet Capture:
 - Click on the selected interface to start capturing packets.

Step 3: Simulating an Attack

1. Initiate a Malicious Activity:
 - For demonstration purposes, simulate an attack by executing a known malware sample or using a penetration testing tool to generate suspicious traffic. Ensure this is done in a controlled and legal environment.

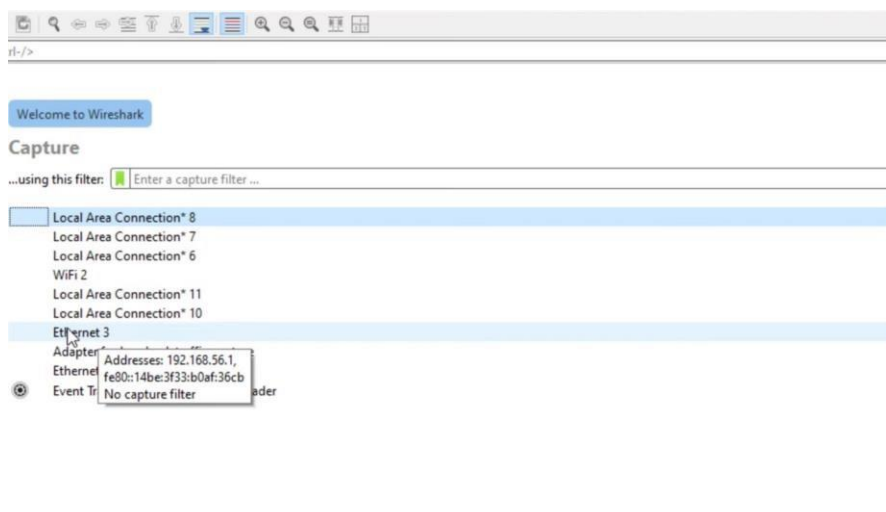
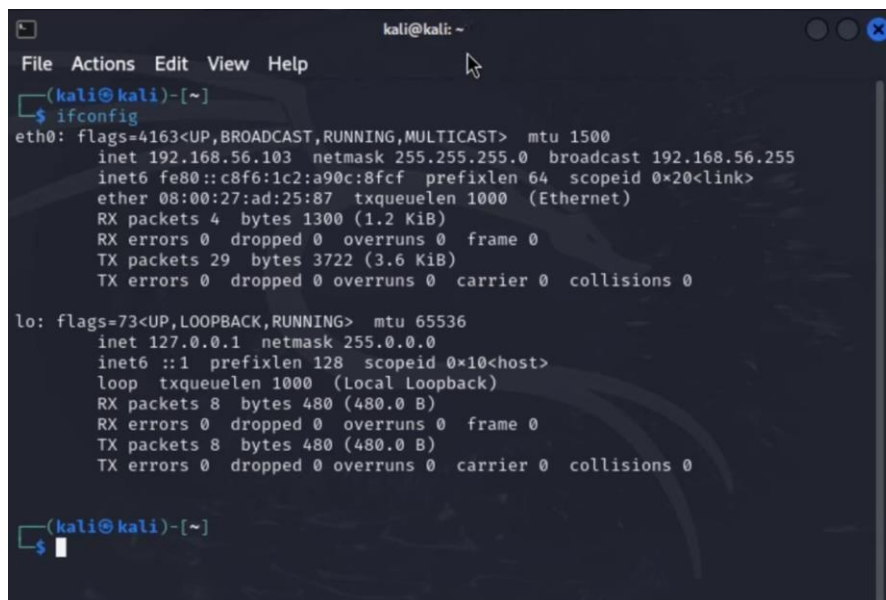
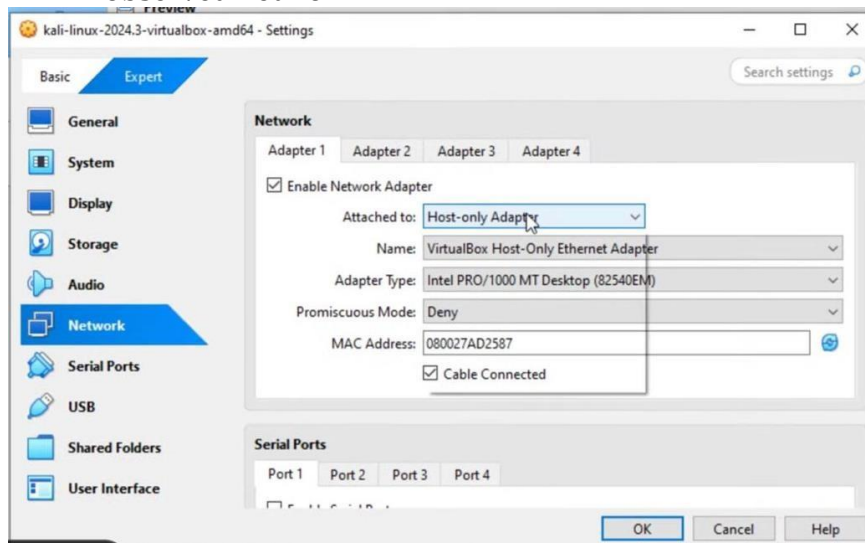
Step 4: Analyzing Captured Traffic

1. Stop Packet Capture:
 - After the simulation, stop the capture in Wireshark.
2. Apply Filters:
 - Use Wireshark filters to isolate suspicious traffic. For example:
 - `ip.addr == [attacker_ip]` to filter traffic from a specific IP address.
 - `http.request` to view HTTP requests.
3. Examine Packets:
 - Inspect the details of filtered packets to identify anomalies, such as unusual protocols, unexpected destinations, or data patterns indicative of malicious activity.

Observations: Indicators of Compromise

- Unusual Network Connections:
 - Connections to known malicious IP addresses or domains.
- Data Exfiltration:
 - Large volumes of outbound traffic, especially to unfamiliar external addresses.

- Suspicious Payloads:
 - Payloads containing encoded or encrypted data that are not typical for the observed network.



Wireshark packet capture analysis showing ICMP Echo (ping) requests and replies between 192.168.56.1 and 192.168.56.103. The capture is filtered by 'icmp'.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000000	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) request id=0x7905, seq=1/256, ttl=128 (request in 1)
3	1.003904	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) reply id=0x7905, seq=2/512, ttl=64 (reply in 4)
4	1.004246	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) request id=0x7905, seq=2/512, ttl=128 (request in 3)
5	2.004512	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) reply id=0x7905, seq=3/768, ttl=64 (reply in 6)
6	2.004774	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) request id=0x7905, seq=3/768, ttl=128 (request in 5)
7	3.020104	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) reply id=0x7905, seq=4/1024, ttl=64 (reply in 8)
8	3.020359	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) request id=0x7905, seq=4/1024, ttl=128 (request in 7)
9	4.020318	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) reply id=0x7905, seq=5/1280, ttl=64 (reply in 10)
10	4.020508	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) request id=0x7905, seq=5/1280, ttl=128 (request in 9)
11	4.553353	0a:00:27:00:00:0d	PCSSystemtec_ad:251...	ARP	42	Who has 192.168.56.103? Tell 192.168.56.1
12	4.553601	PCSSystemtec_ad:251...	0a:00:27:00:00:0d	ARP	60	192.168.56.103 is at 08:00:27:ad:25:87
13	5.036723	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) request id=0x7905, seq=6/1536, ttl=64 (reply in 14)
14	5.036979	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) reply id=0x7905, seq=6/1536, ttl=128 (request in 13)
15	5.228203	PCSSystemtec_ad:251...	0a:00:27:00:00:0d	ARP	60	Who has 192.168.56.1? Tell 192.168.56.103
16	5.228246	0a:00:27:00:00:0d	PCSSystemtec_ad:251...	ARP	42	192.168.56.1 is at 0a:00:27:00:00:0d
17	6.037813	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) request id=0x7905, seq=7/1792, ttl=64 (reply in 18)
18	6.038074	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) reply id=0x7905, seq=7/1792, ttl=128 (request in 17)
19	7.052375	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) request id=0x7905, seq=8/2048, ttl=64 (reply in 20)
20	7.052556	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) reply id=0x7905, seq=8/2048, ttl=128 (request in 19)
21	8.076715	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) request id=0x7905, seq=9/2304, ttl=64 (reply in 22)
22	8.077004	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) reply id=0x7905, seq=9/2304, ttl=128 (request in 21)
23	9.077458	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) request id=0x7905, seq=10/2560, ttl=64 (reply in 24)
24	9.077625	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) reply id=0x7905, seq=10/2560, ttl=128 (request in 23)
25	10.092129	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) request id=0x7905, seq=11/2816, ttl=64 (reply in 26)
26	10.092382	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) reply id=0x7905, seq=11/2816, ttl=128 (request in 25)
27	11.116098	192.168.56.103	192.168.56.1	ICMP	98	Echo (ping) request id=0x7905, seq=12/3072, ttl=64 (reply in 28)
28	11.116358	192.168.56.1	192.168.56.103	ICMP	98	Echo (ping) reply id=0x7905, seq=12/3072, ttl=128 (request in 27)

Packet 11 details: Ethernet II, Src: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87), Dst: 0a:00:27:00:00:0d (0a:00:27:00:00:0d), Internet Protocol Version 4, Src: 192.168.56.103, Dst: 192.168.56.1, Internet Control Message Protocol.

```

4 bytes from 192.168.56.1: icmp_seq=9 ttl=128 time=0.938 ms
4 bytes from 192.168.56.1: icmp_seq=10 ttl=128 time=0.471 ms
4 bytes from 192.168.56.1: icmp_seq=11 ttl=128 time=0.680 ms
4 bytes from 192.168.56.1: icmp_seq=12 ttl=128 time=0.696 ms
4 bytes from 192.168.56.1: icmp_seq=13 ttl=128 time=0.560 ms
4 bytes from 192.168.56.1: icmp_seq=14 ttl=128 time=0.774 ms
4 bytes from 192.168.56.1: icmp_seq=15 ttl=128 time=0.888 ms
4 bytes from 192.168.56.1: icmp_seq=16 ttl=128 time=0.746 ms
4 bytes from 192.168.56.1: icmp_seq=17 ttl=128 time=0.723 ms
4 bytes from 192.168.56.1: icmp_seq=18 ttl=128 time=0.702 ms
4 bytes from 192.168.56.1: icmp_seq=19 ttl=128 time=0.698 ms
4 bytes from 192.168.56.1: icmp_seq=20 ttl=128 time=0.702 ms
4 bytes from 192.168.56.1: icmp_seq=21 ttl=128 time=0.382 ms
4 bytes from 192.168.56.1: icmp_seq=22 ttl=128 time=0.624 ms
4 bytes from 192.168.56.1: icmp_seq=23 ttl=128 time=0.820 ms
4 bytes from 192.168.56.1: icmp_seq=24 ttl=128 time=0.449 ms
4 bytes from 192.168.56.1: icmp_seq=25 ttl=128 time=0.849 ms
4 bytes from 192.168.56.1: icmp_seq=26 ttl=128 time=0.981 ms
C
-- 192.168.56.1 ping statistics --
6 packets transmitted, 26 received, 0% packet loss, time 25301ms
rtt min/avg/max/mdev = 0.256/0.689/1.026/0.175 ms

--(kali@kali)-[~]
$ ping -i 0.002 192.168.56.1

```

```

File Actions Edit View Help

--(root@kali)-[~]
# nmap -sS 192.168.56.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-17 23:13 IST
Nmap scan report for 192.168.56.1
Host is up (0.00041s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
7070/tcp  open  realserver
MAC Address: 0A:00:27:00:00:0F (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 30.11 seconds

--(root@kali)-[~]
#

```

Wireshark - Packet 12 - Ethernet 3

Frame 12: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{7868CD3...}

Ethernet II, Src: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87), Dst: 0a:00:27:00:00:0d (0a:00:27:00:00:0d)

Internet Protocol Version 4, Src: 192.168.56.103, Dst: 192.168.56.1

Transmission Control Protocol, Src Port: 60244, Dst Port: 80, Seq: 0, Len: 0

Ethernet 3

Filter: p.src==192.168.56.103

Time	Source	Destination	Protocol	Length	Info
1464.28.866810	192.168.56.103	192.168.56.1	TCP	60	60244 → 9535 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1465.28.869725	192.168.56.103	192.168.56.1	TCP	60	60244 → 1090 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1466.28.876646	192.168.56.103	192.168.56.1	TCP	60	60244 → 2111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1467.28.879562	192.168.56.103	192.168.56.1	TCP	60	60244 → 8649 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1468.28.885364	192.168.56.103	192.168.56.1	TCP	60	60244 → 7070 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1470.28.885764	192.168.56.103	192.168.56.1	TCP	60	60244 → 7070 [RST] Seq=1 Win=0 Len=0
1471.28.888293	192.168.56.103	192.168.56.1	TCP	60	60244 → 563 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1472.28.888333	192.168.56.103	192.168.56.1	TCP	60	60244 → 38292 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1473.28.888362	192.168.56.103	192.168.56.1	TCP	60	60244 → 2383 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1474.28.888499	192.168.56.103	192.168.56.1	TCP	60	60244 → 64680 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1475.28.888534	192.168.56.103	192.168.56.1	TCP	60	60244 → 691 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1476.28.888562	192.168.56.103	192.168.56.1	TCP	60	60244 → 4004 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1477.28.888714	192.168.56.103	192.168.56.1	TCP	60	60244 → 1097 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1478.28.888783	192.168.56.103	192.168.56.1	TCP	60	60244 → 32788 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1479.28.888817	192.168.56.103	192.168.56.1	TCP	60	60244 → 1062 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1480.28.888944	192.168.56.103	192.168.56.1	TCP	60	60244 → 1277 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1481.28.888984	192.168.56.103	192.168.56.1	TCP	60	60244 → 8180 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1482.28.889012	192.168.56.103	192.168.56.1	TCP	60	60244 → 5850 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1483.28.889262	192.168.56.103	192.168.56.1	TCP	60	60244 → 9502 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1484.28.889297	192.168.56.103	192.168.56.1	TCP	60	60244 → 4005 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1485.28.889329	192.168.56.103	192.168.56.1	TCP	60	60244 → 7921 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1486.28.889441	192.168.56.103	192.168.56.1	TCP	60	60244 → 1108 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1487.28.889480	192.168.56.103	192.168.56.1	TCP	60	60244 → 90 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1488.28.889507	192.168.56.103	192.168.56.1	TCP	60	60244 → 34571 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1489.28.889636	192.168.56.103	192.168.56.1	TCP	60	60244 → 1058 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1490.28.889671	192.168.56.103	192.168.56.1	TCP	60	60244 → 1071 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1491.28.889699	192.168.56.103	192.168.56.1	TCP	60	60244 → 5102 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
1492.28.889844	192.168.56.103	192.168.56.1	TCP	60	60244 → 5051 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Frame 16: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{...}

Ethernet II, Src: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87), Dst: 0a:00:27:00:00:0d (0a:00:27:00:00:0d)

Internet Protocol Version 4, Src: 192.168.56.103, Dst: 192.168.56.1

Transmission Control Protocol, Src Port: 60244, Dst Port: 80, Seq: 0, Len: 0

Conclusion

This practical demonstrates the importance of network traffic analysis in identifying security incidents on Windows hosts. By capturing and examining packets with Wireshark, one can detect signs of compromise, understand the nature of the attack, and implement appropriate remediation measures to enhance system security.

Part C: Investigating a Malware Exploit

Objective:

The objective of this practical is to investigate a malware exploit using Kali Linux by analyzing a suspicious file. This experiment demonstrates how to identify malicious activities by examining the file's contents and behavior, providing insights into potential security threats.

Introduction

In cybersecurity, analyzing suspicious files is crucial for detecting and understanding malware. Attackers often use various techniques to obfuscate malicious code, making it essential to perform in-depth analysis. By utilizing tools available in Kali Linux, security professionals can uncover hidden threats within files, assess their behavior, and determine the appropriate mitigation strategies.

Commands and Steps

Step 1: Setting Up the Environment

1. Download the Suspicious File
 - Obtain a sample malware file from a reputable source for analysis.

Step 2: Calculating the File's SHA-256 Hash

1. Open Terminal
 - Launch the terminal in Kali Linux.
2. Navigate to the File's Directory
 - Use the `cd` command to go to the directory containing the suspicious file:
`cd /path/to/file`
3. Calculate the SHA-256 Hash
 - Run the following command to generate the hash:
`sha256sum filename`
 - Replace filename with the actual name of the suspicious file.

Step 3: Analyzing the File with Strings

1. Use the strings Command
 - Extract readable strings from the file:
`strings filename`
 - Review the output for any suspicious or notable text.

Step 4: Examining the File with a Hex Editor

1. Install HexEditor (if not already installed)
 - Run the installation command:
`sudo apt-get install hexedit`
2. Open the File in HexEditor
 - Run the command:
`hexedit filename`
 - Navigate through the file to inspect its hexadecimal and ASCII representations for anomalies.

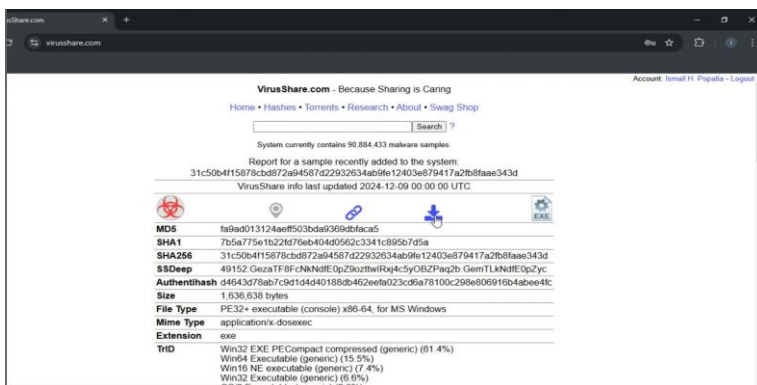
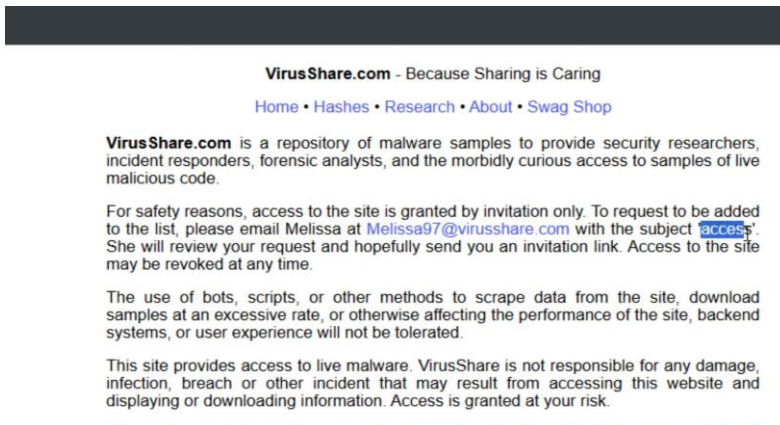
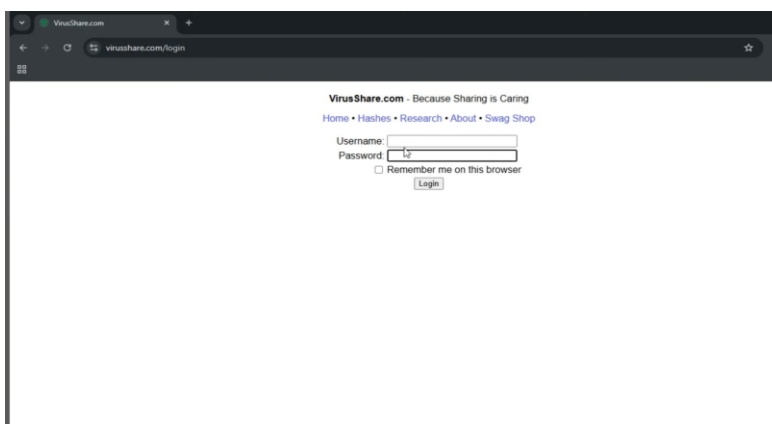
Step 5: Checking the File on VirusTotal

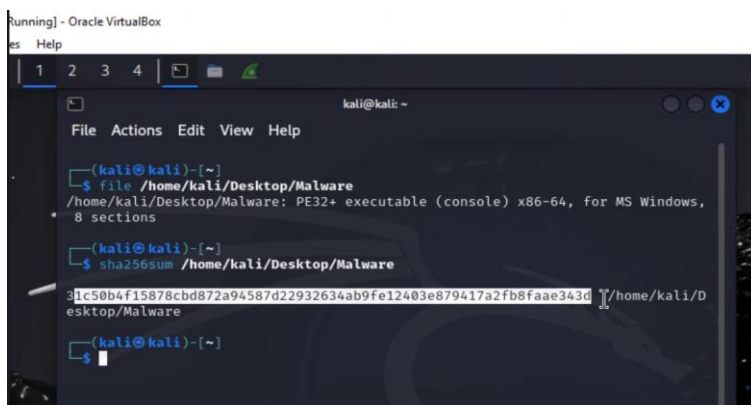
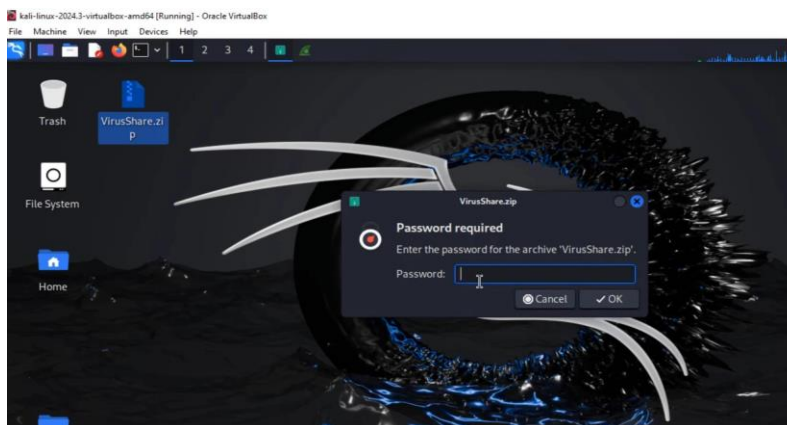
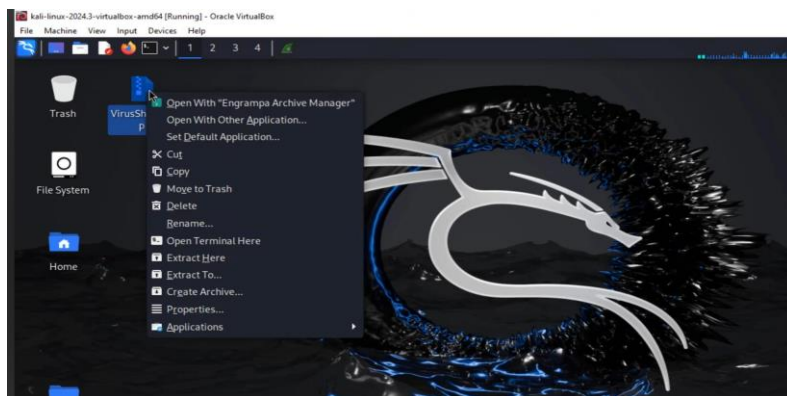
1. Access VirusTotal
 - Visit [VirusTotal](https://www.virustotal.com) in your web browser.
2. Upload the File
 - Click on "Choose file" and select the suspicious file to upload.
3. Analyze the Results

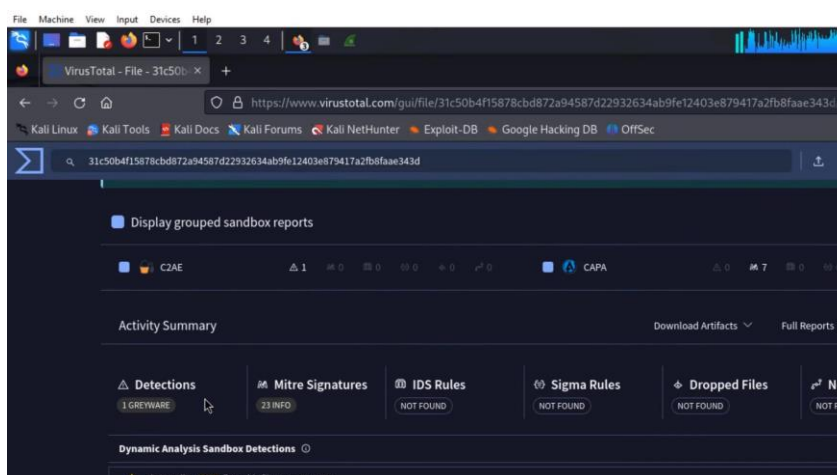
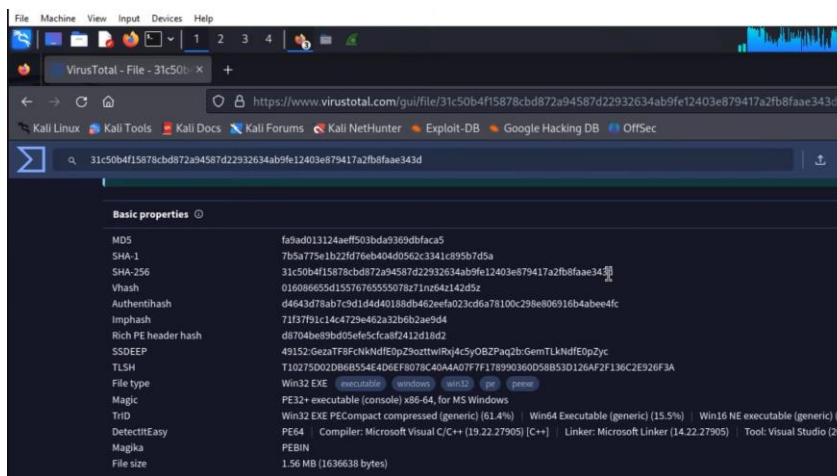
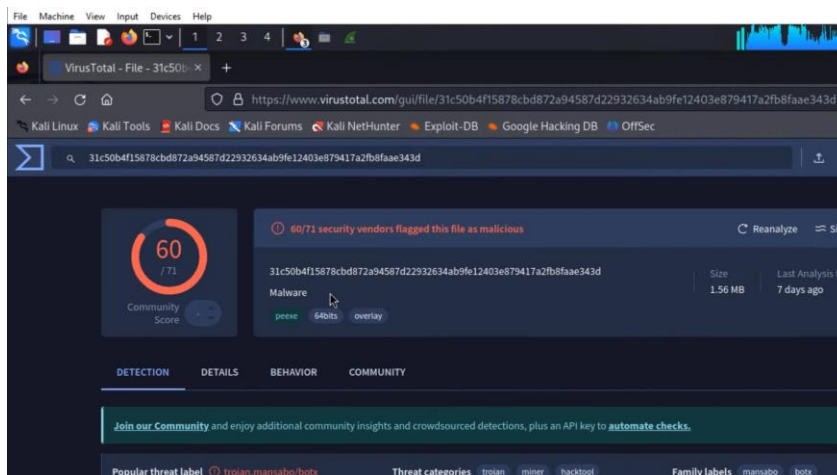
- Review the analysis provided by VirusTotal to see if the file is detected as malicious by any antivirus engines.

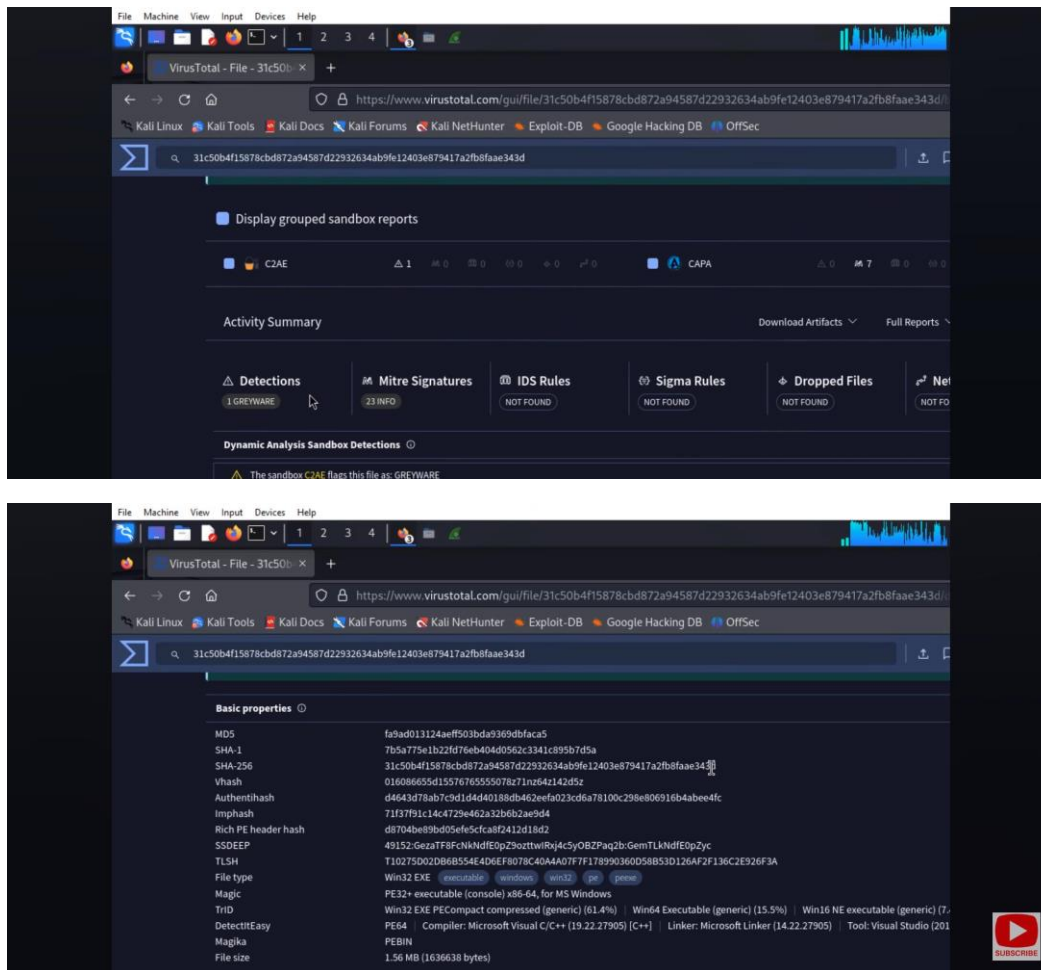
Observations: Indicators of Compromise

- Suspicious Strings
 - Readable text within the file that indicates malicious intent, such as URLs, IP addresses, or specific commands.
- Anomalous Hex Values
 - Unusual patterns or data within the file's hexadecimal representation that deviate from standard file structures.
- VirusTotal Detections
 - Multiple antivirus engines flagging the file as malicious, confirming its potential threat.









Conclusion

This practical demonstrates the importance of file analysis in identifying malware exploits. By utilizing tools in Kali Linux to examine a suspicious file's **hash, strings, and hexadecimal content**, and by cross-referencing with **VirusTotal**, one can detect signs of compromise, understand the nature of the threat, and implement appropriate remediation measures to enhance system security.

Practical No: 3

Part A: Demonstrate the use of Snort and Firewall Rules

Objective

The objective of this practical is to demonstrate the use of Snort, an open-source network intrusion detection and prevention system (IDS/IPS), and firewall rules to monitor and control network traffic. This experiment showcases how to set up Snort to detect suspicious activity and configure firewall rules to block malicious traffic.

Introduction

Snort – Intrusion Detection and Prevention System (IDS/IPS)

Snort is a powerful tool that can analyze network packets in real-time to detect potential threats. It can operate in different modes:

1. Sniffer Mode – Captures and displays network traffic.
2. Packet Logger Mode – Logs network traffic to a file for later analysis.
3. Network Intrusion Detection Mode – Uses predefined rules to detect and alert on malicious activity.

Firewall Rules

Firewalls are used to control incoming and outgoing network traffic based on security rules. Linux-based systems commonly use iptables or UFW (Uncomplicated Firewall) to manage firewall rules.

Commands and Steps

Step 1: Installing Snort

1. Update the System:
`sudo apt update && sudo apt upgrade -y`
2. Install Snort:
`sudo apt install snort -y`
3. Verify Snort Installation:
`snort -V`
 - This will display the installed version of Snort.

Step 2: Running Snort in Different Modes

1. Sniffer Mode (Monitor Live Traffic):
`sudo snort -v -i eth0`
 - -v: Enables verbose output.
 - -i eth0: Specifies the network interface (change eth0 as needed).
2. Packet Logger Mode (Save Traffic for Analysis):
`sudo snort -dev -i eth0 -l /var/log/snort/`
 - -d: Shows application layer data.
 - -e: Displays Ethernet headers.
 - -v: Verbose mode.
 - -l /var/log/snort/: Logs traffic to the specified directory.
3. Network Intrusion Detection Mode (Using Predefined Rules):
`sudo snort -c /etc/snort/snort.conf -i eth0`
 - -c: Specifies the configuration file that contains Snort rules.
 - -i eth0: Defines the network interface.

Step 3: Writing a Basic Snort Rule

1. Open Snort Rules File:
`sudo nano /etc/snort/rules/local.rules`
2. Add a Simple Rule to Detect ICMP (Ping) Traffic:
`alert icmp any any -> any any (msg:"ICMP detected"; sid:1000001; rev:1;)`
 - This rule alerts when any ICMP (ping) packet is detected.
3. Save and Exit:
 - Press CTRL+X, then Y, and ENTER to save the file.
4. Test the Rule by Running Snort:
`sudo snort -c /etc/snort/snort.conf -i eth0`
5. Trigger the Rule by Sending a Ping Request:
`ping -c 4 <target-ip>`
 - Replace <target-ip> with an actual IP address.
6. Check Snort Alerts:
`cat /var/log/snort/alert`

Step 4: Configuring Firewall Rules (Using UFW & iptables)

A. Using UFW (Uncomplicated Firewall)

1. Enable UFW:
`sudo ufw enable`
2. Allow SSH Traffic:
`sudo ufw allow 22/tcp`
3. Block Incoming ICMP (Ping) Requests:
`sudo ufw deny proto icmp from any to any`
4. Allow Web Traffic (HTTP & HTTPS):
`sudo ufw allow 80/tcp`
`sudo ufw allow 443/tcp`
5. Check Firewall Status:
`sudo ufw status`

B. Using iptables

1. List Current iptables Rules:
`sudo iptables -L -v`
2. Block All Incoming Traffic from a Specific IP (Replace <IP> with an actual address):
`sudo iptables -A INPUT -s <IP> -j DROP`
3. Allow SSH Access from a Specific IP:
`sudo iptables -A INPUT -p tcp --dport 22 -s <IP> -j ACCEPT`
4. Drop All Incoming ICMP Requests (Block Ping):
`sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP`
5. Save the iptables Rules Permanently:
`sudo iptables-save > /etc/iptables/rules.v4`

Observations: Security Enhancements

- Snort IDS Alerting on ICMP Traffic:
 - When a ping is sent, Snort logs an alert as per the rule configured.
- Firewall Blocking Malicious Traffic:
 - Attempts to ping or access blocked ports are rejected, improving security.
- Preventing Unauthorized Access:

- Restricting SSH access and blocking suspicious IPs strengthens security posture.

Conclusion

This practical demonstrates how Snort and firewall rules can be used to monitor and secure a network. Snort effectively detects and logs suspicious traffic, while firewall rules help block unwanted connections and protect the system from attacks. By implementing both, security professionals can create a strong defense against network intrusions.

Part B: Demonstrate Extract an Executable from a PCAP

Objective

The objective of this practical is to extract an executable file from a PCAP (Packet Capture) file using Wireshark and other tools. This experiment demonstrates how network traffic, especially file transfers over protocols like HTTP, FTP, or SMB, can contain executables that may be extracted and analyzed for potential malware.

Introduction

PCAP File Analysis

A PCAP file captures network traffic, which can contain various types of data, including executables. These executables may be transferred over the network using protocols like HTTP, FTP, or even SMB. Extracting files from PCAPs is crucial for cybersecurity professionals who want to analyze suspicious network activity or conduct malware investigations.

File Transfer Protocols

During file transfers, executables are often embedded within network packets. Tools like Wireshark can help capture and inspect network traffic to find these files. By extracting the files from the captured packets, we can analyze them for potential threats.

Tools Used

- Wireshark: A network packet analyzer to capture and inspect network traffic.
- NetworkMiner: A network forensics tool that helps to extract files from network captures.
- tcpdump: A command-line tool used for capturing network traffic, often used in conjunction with Wireshark.
- Strings: A command-line tool to extract readable strings from binary files (like executables).

Steps

Step 1: Open the PCAP File in Wireshark

1. Launch Wireshark on your machine.
2. Open the captured PCAP file you want to analyze using File > Open.
3. Inspect the PCAP to find the network traffic that could be carrying the executable. Common protocols to look for are HTTP, FTP, or SMB.

Step 2: Filter for Relevant Traffic

1. Use the appropriate filter to identify the file transfer protocol. For example, to filter for HTTP traffic, use:
http
Or for FTP traffic:
ftp
2. Inspect the packet details to locate the packet containing the executable transfer. In HTTP traffic, this could be a POST or GET request with an attached file.

Step 3: Extract the Executable Using Wireshark

1. Find the data section of the HTTP or FTP packet that contains the executable file. This is typically located in the Payload of the packet.
2. Right-click the relevant packet in Wireshark.
3. Select Follow > TCP Stream to view the entire data exchange.
4. Save the stream to a file by clicking on the Save As button. Ensure the file extension is appropriate (e.g., .exe for Windows executables).

Step 4: Analyze the Extracted Executable

1. After extracting the file, use a tool like strings to inspect the contents of the executable for any readable strings:
strings extracted_file.exe
2. Alternatively, you can run the extracted file through a malware analysis tool like VirusTotal or use a sandbox environment to test the behavior of the file.

Step 5: Additional Verification (Using NetworkMiner)

1. Download and install NetworkMiner.
2. Open the PCAP file in NetworkMiner.
3. Navigate to the Files tab to see all the files extracted from the network traffic.
4. Locate the executable file and click to extract it to your local system for further analysis.

Observation: Why Extracting Executables is Important

Extracting executables from network traffic allows security analysts to:

- Detect malicious files that are being transferred over the network.
- Examine potentially harmful files without executing them on the local machine.
- Gain insight into how malware spreads through a network.

Conclusion

This practical demonstrates how network traffic, captured in PCAP files, can be analyzed to extract executable files. By using Wireshark, tcpdump, or NetworkMiner, we can detect and extract files that may be used for malicious purposes. Extracting executables from network traffic is an important part of network security analysis and can help in identifying and mitigating security threats.

Part C: Demonstrate a practical for Exploring DNS Traffic

Objective

The objective of this practical is to explore DNS (Domain Name System) traffic using Wireshark. This experiment demonstrates how DNS queries and responses can be analyzed to monitor network activities, detect potential security issues, and troubleshoot DNS-related problems.

Introduction

DNS Overview

DNS is a protocol used to resolve domain names into IP addresses, allowing users to access websites using human-readable names. Every time a user accesses a website, their device sends a DNS query to a DNS server to resolve the domain name into an IP address. This DNS traffic can be captured and analyzed for various purposes, including monitoring DNS requests, identifying suspicious queries, and ensuring network security.

Analyzing DNS Traffic

DNS traffic, although often overlooked, can provide valuable insights into network activities. It is important to analyze DNS queries to detect any malicious activities, such as DNS tunneling (using DNS to exfiltrate data) or querying for suspicious domains.

Tools Used

- Wireshark: A network protocol analyzer used to capture and inspect DNS traffic.
- Tcpcat: A command-line packet capture tool (optional).
- DNS Server: A local or public DNS server to generate DNS traffic.

Steps

Step 1: Set Up the Environment

1. Ensure that you have Wireshark installed on your machine.
2. Ensure that the machine or network you are working on has an active internet connection or DNS server to generate traffic. Alternatively, you can set up a DNS server locally.

Step 2: Capture DNS Traffic with Wireshark

1. Launch Wireshark on your machine.
2. Select the network interface you want to capture traffic on (e.g., Wi-Fi, Ethernet).
3. Start packet capturing by clicking on the Start capturing packets button.
4. To focus specifically on DNS traffic, apply the following filter in the Wireshark filter bar:
dns

This will filter out all non-DNS packets, allowing you to only see DNS queries and responses.

Step 3: Generate DNS Traffic

1. Open a web browser or use any application that accesses the internet, such as ping, nslookup, or dig.
2. To generate DNS queries manually, you can use a command like:
nslookup google.com

This will send a DNS query for google.com to the DNS server.

Alternatively, you can simply browse websites to generate DNS traffic.

Step 4: Analyze DNS Queries in Wireshark

1. Once you have generated DNS traffic, return to Wireshark to observe the captured packets.
2. Look for packets labeled as DNS in the Protocol column.
3. Click on a DNS packet to examine the details. You should be able to see:
 - DNS Query: The client's request to resolve a domain name.
 - DNS Response: The DNS server's response with the corresponding IP address for the domain.
4. In the Packet Details pane, expand the DNS section to view details such as:
 - Transaction ID: Identifies each DNS transaction.
 - Queries: The domain names requested by the client.
 - Answers: The resolved IP addresses returned by the DNS server.

Step 5: Filtering Specific DNS Traffic

1. Use filters to drill down into specific DNS traffic. For example:
 - To view only DNS queries for a particular domain, use:
arduino
dns.qry.name == "google.com"
 - To view DNS responses, use:
dns.flags.response == 1
2. Observe the query type (e.g., A record, AAAA record, MX record) and response type (e.g., IP address, CNAME, etc.).

Step 6: Detecting Suspicious DNS Traffic

1. Look for any anomalies in the DNS traffic, such as:
 - High Volume of DNS Queries: Unusually high amounts of DNS queries could indicate DNS tunneling or a DDoS attack.
 - Suspicious Domain Names: Check for domains that are uncommon or related to known malicious activities.
 - Unusual Query Types: Look for DNS queries requesting MX (mail exchange) or TXT records, which could be used for data exfiltration.
2. To further analyze DNS traffic for potential threats, you can export specific packets or use external tools like dnsmap or dnsrecon for DNS reconnaissance.

Step 7: Additional Insights and Logs

1. Export DNS Logs: In Wireshark, you can export DNS traffic for further analysis. To export:
 - Go to File > Export Specified Packets.
 - Select the DNS packets to save to a .pcap file.
2. Review the exported data to search for any unusual patterns.

Observation: Importance of DNS Traffic Analysis

- DNS Query Analysis helps monitor and troubleshoot domain resolution issues.
- Malicious Activities such as DNS tunneling or unauthorized domain lookups can be detected.
- Network Performance: DNS latency or failures can be detected and addressed by analyzing DNS queries and responses.

Conclusion

This practical demonstrates the importance of analyzing DNS traffic. By using tools like Wireshark, you can capture and inspect DNS queries and responses, which helps in troubleshooting, network performance optimization, and detecting potential security threats such as DNS tunneling. Understanding how DNS operates in a network is critical for network administrators and cybersecurity professionals.

Practical No: 4

Part A: Using Wireshark to Examine HTTP and HTTPS Traffic

Objective

The objective of this practical is to demonstrate how to capture and analyze HTTP and HTTPS traffic using Wireshark. This experiment will show how HTTP traffic is transmitted in plaintext, while HTTPS traffic is encrypted, ensuring secure communication.

Introduction

HTTP Overview

HTTP (HyperText Transfer Protocol) is used to transfer data over the web, such as loading web pages. It is unencrypted, meaning all data transmitted, including usernames, passwords, and sensitive information, is in plaintext. HTTPS (HyperText Transfer Protocol Secure) is the encrypted version of HTTP. It uses SSL/TLS protocols to secure the connection, making it much harder for attackers to intercept or read the data transmitted between the client and server.

Tools Used

- Wireshark: A network protocol analyzer that captures and inspects network packets.
- Web Browser: To generate HTTP/HTTPS traffic.
- SSL/TLS certificates: For HTTPS traffic decryption (optional).

Steps

Step 1: Set Up Wireshark for Packet Capture

1. Install Wireshark: Ensure you have Wireshark installed on your system.
2. Select the Network Interface:
 - Open Wireshark.
 - Select the network interface (Wi-Fi or Ethernet) you will use to capture the traffic.
3. Start Capturing:
 - Click the Start capturing packets button (the shark fin icon).
 - Begin capturing the traffic.

Step 2: Filter HTTP and HTTPS Traffic

1. Set Filters for HTTP:
 - To capture only HTTP traffic, use the filter:
`http`
 - This will show you packets related to the HTTP protocol.
2. Set Filters for HTTPS:
 - Since HTTPS encrypts its traffic, you can filter by the port used for HTTPS (usually port 443):
`tcp.port == 443`
 - This will capture packets on the HTTPS port.

Step 3: Generate HTTP and HTTPS Traffic

1. Generate HTTP Traffic:

- Open a web browser and visit a non-HTTPS website, such as <http://example.com>.
- This will generate unencrypted HTTP traffic, which you will see in Wireshark.

2. Generate HTTPS Traffic:

- Open a web browser and visit a secure website like <https://example.com>.
- This will generate encrypted HTTPS traffic.

Step 4: Analyze HTTP Traffic

1. Inspect HTTP Packets:

- In Wireshark, you will see various HTTP packets. Look for packets with the HTTP protocol in the Protocol column.
- Click on an HTTP packet to inspect it.
- Under the Packet Details pane, you will see the HTTP request and response, including:
 - HTTP Request: URL, method (GET, POST, etc.), headers, and parameters.
 - HTTP Response: Status code, content length, and headers.

2. Observe Unencrypted Data:

- Since HTTP is unencrypted, all the data is visible, including any credentials or sensitive information sent in the request (e.g., login details, form submissions).

Step 5: Analyze HTTPS Traffic

1. Inspect HTTPS Packets:

- Since HTTPS traffic is encrypted, you won't be able to see the content of the HTTPS request and response. All you will see in the packet details is the initial TLS handshake and the encrypted data.

2. Understanding the TLS Handshake:

- The TLS handshake is visible in the captured packets. This process involves:
 - Client Hello: The client (browser) initiates the connection by offering supported cipher suites.
 - Server Hello: The server responds with its chosen cipher suite and SSL/TLS certificate.
 - Key Exchange: A secure key exchange takes place to establish an encrypted session.

3. Decryption (Optional):

- If you want to decrypt HTTPS traffic in Wireshark:
 - Obtain the Server's Private Key (if possible) or use SSL Key Log (if using a browser like Firefox or Chrome that can log the session keys).
 - In Wireshark, go to Edit > Preferences.
 - Under Protocols, select SSL and input the server's private key or SSL key log file.

Step 6: Filter and Export Traffic

1. Export HTTP/HTTPS Packets:

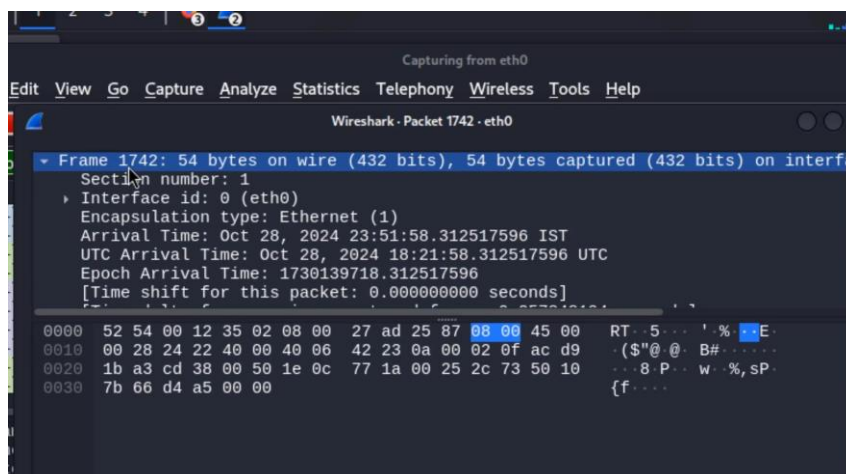
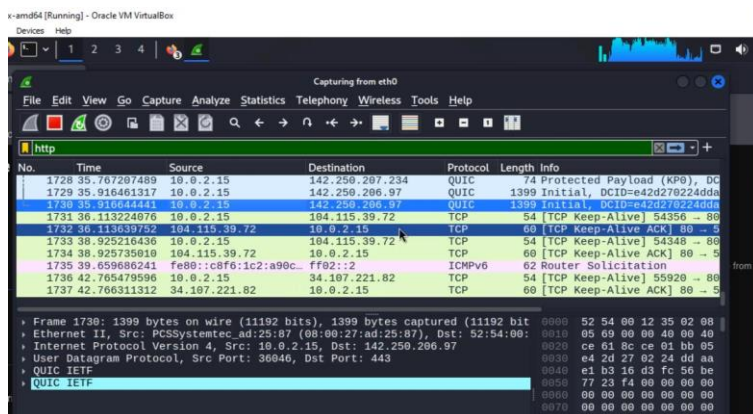
- After capturing the traffic, you can export the packets for later analysis. Go to File > Export Specified Packets.
- Choose the packets you want to export, and save them as a .pcap file.

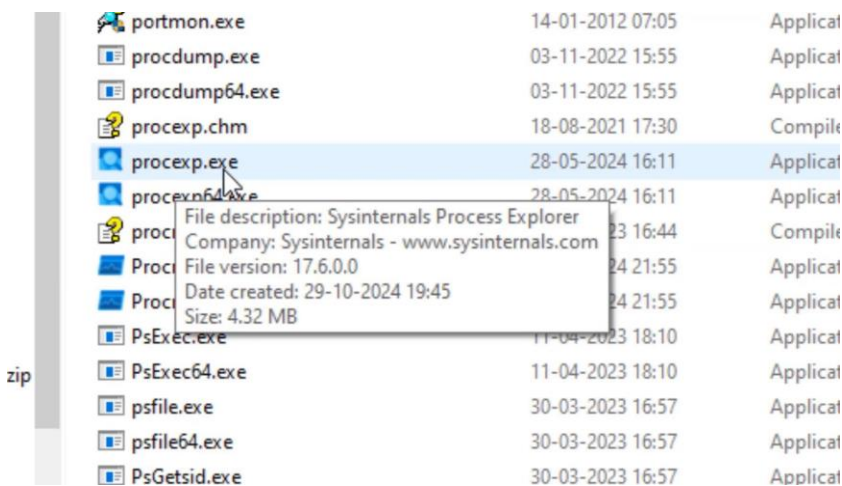
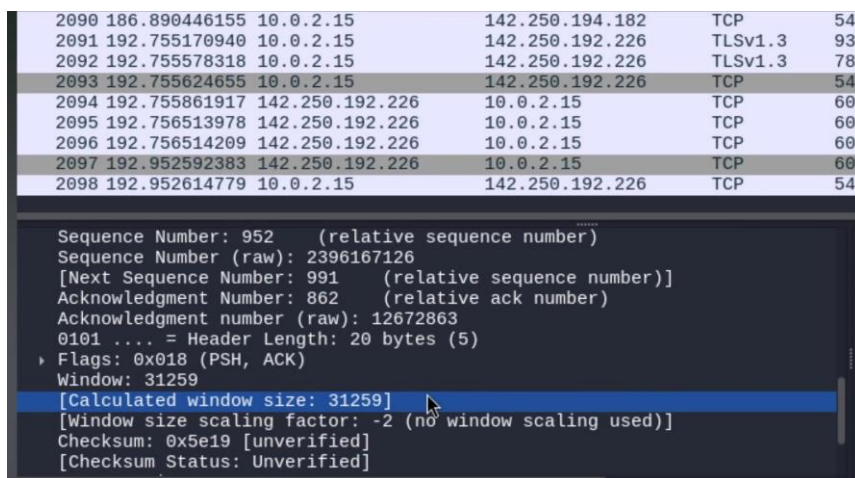
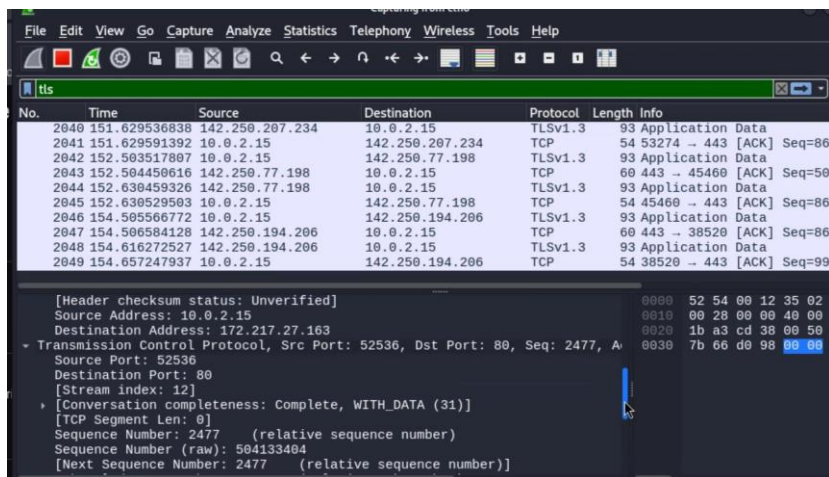
2. Apply Display Filters:

- You can apply display filters to narrow down the packets you're interested in. For example, to view only HTTP requests: `http.request`
- To view only HTTPS handshake packets: `ssl.handshake`

Step 7: Observation and Conclusion

- HTTP Traffic:
 - All data transmitted is visible in plaintext (including credentials), making it insecure for sensitive transactions.
- HTTPS Traffic:
 - The data is encrypted and not readable in the packet capture, providing confidentiality and protection against eavesdropping.
- Security Implications:
 - HTTPS ensures secure communication by preventing third parties from intercepting or altering the data.
 - HTTP should be avoided for sensitive transactions as it exposes all information in the request and response.





Process Explorer - Sysinternals: www.sysinternals.com [DESKTOP-QVURW7USER] (Administrator)

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
Registry	6.744 K	82,876 K	108			
System Idle Process	71.57	60 K	8 K	0		
System	< 0.01	204 K	3,460 K	4		
smss.exe	0.74	0 K	0 K	n/a	Hardware Interrupts and DPCs	
svchost.exe	< 0.01	1,068 K	1,048 K	416		
csrss.exe	316 K	33,020 K	1956			
csrss.exe	2,096 K	5,524 K	592			
csrss.exe	1,404 K	6,748 K	684			
csrss.exe	5,512 K	9,800 K	688			
svchost.exe	0.37	11,596 K	28,700 K	956	Host Process for Windows S...	Microsoft Corporation
csrss.exe	3,304 K	11,068 K	5634	COM Surrogate		Microsoft Corporation
csrss.exe	20,186 K	62,760 K	1800			
csrss.exe	6,048 K	26,368 K	7516	Runtime Broker		Microsoft Corporation
csrss.exe	2,10,552 K	2,30,816 K	7580	Search application		Microsoft Corporation
csrss.exe	18,400 K	33,128 K	7524	Runtime Broker		Microsoft Corporation
csrss.exe	18,400 K	64,000 K	3344	Search application		Microsoft Corporation
csrss.exe	31,320 K	52,864 K	8524	Microsoft Edge WebView2		Microsoft Corporation
csrss.exe	2,176 K	6,444 K	8506	Microsoft Edge WebView2		Microsoft Corporation
csrss.exe	44,372 K	58,228 K	8592	Microsoft Edge WebView2		Microsoft Corporation
csrss.exe	11,076 K	31,708 K	8592	Microsoft Edge WebView2		Microsoft Corporation
csrss.exe	7,868 K	17,384 K	8212	Microsoft Edge WebView2		Microsoft Corporation
csrss.exe	38,220 K	82,092 K	8244	Microsoft Edge WebView2		Microsoft Corporation
csrss.exe	7,868 K	22,848 K	8532	Runtime Broker		Microsoft Corporation
csrss.exe	18,612 K	56,892 K	8534	LockApp.exe		Microsoft Corporation
csrss.exe	9,780 K	38,300 K	8534	Runtime Broker		Microsoft Corporation
csrss.exe	15,448 K	41,160 K	1796			
csrss.exe	24,692 K	3,096 K	8234	Settings		Microsoft Corporation
csrss.exe	12,236 K	33,260 K	8240	Application Frame Host		Microsoft Corporation
csrss.exe	1,852 K	9,388 K	9360	User I/OSSB Broker		Microsoft Corporation
csrss.exe	6,360 K	15,488 K	2276	COM Surrogate		Microsoft Corporation
csrss.exe	19,460 K	61,154 K	8530	Microsoft Edge WebView2		Microsoft Corporation
csrss.exe	4,164 K	24,100 K	8544	Runtime Broker		Microsoft Corporation
csrss.exe	52,662 K	1,58,116 K	11524	Microsoft Phone Link		Microsoft Corporation

Process Explorer - Sysinternals: www.sysinternals.com [DESKTOP-QVURW7USER] (Administrator)

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
Registry	75.08	6,760 K	82,892 K	108		
System Idle Process	1.01	60 K	8 K	0		
System	0.67	204 K	3,460 K	4		
smss.exe	1,068	0 K	0 K	n/a	Hardware Interrupts and DPCs	
svchost.exe	< 0.01	2,444 K	316			
csrss.exe	< 0.01	1,404 K	3,460 K	4		
csrss.exe	< 0.01	2,708 K	5,458 K	592		
csrss.exe	< 0.01	1,404 K	6,664 K	684		
csrss.exe	< 0.01	2,780 K	7,148 K	692		
csrss.exe	< 0.01	3,800 K	10,524 K	784	Windows Log-on Application	Microsoft Corporation
csrss.exe	< 0.01	3,500 K	12,788 K	5816	gfm.exe Module	Intel Corporation
csrss.exe	< 0.01	2,560 K	9,876 K	5836	gfm.exe Module	Intel Corporation
csrss.exe	< 0.01	3,008 K	11,432 K	5856		
csrss.exe	< 0.01	1,06,468 K	1,71,272 K	5036	Windows Explorer	Microsoft Corporation
csrss.exe	< 0.01	1,708 K	9,156 K	8540	Windows Security notification icon	Microsoft Corporation
csrss.exe	< 0.01	5,154 K	13,496 K	10104	Microsoft Office Document Cache	Microsoft Corporation
csrss.exe	< 0.01	29,588 K	16,112 K	10452	AnyDesk	AnyDesk Software GmbH
csrss.exe	< 0.01	6,15,608 K	6,59,176 K	12128	TechSmith Camtasia 2019	TechSmith Corporation

Handles - DLLs - Threads

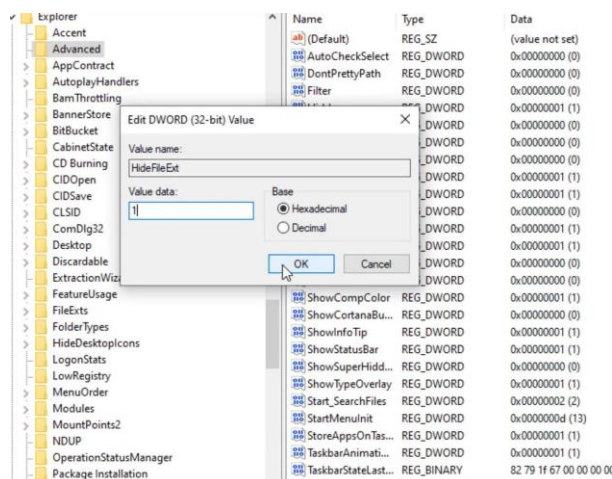
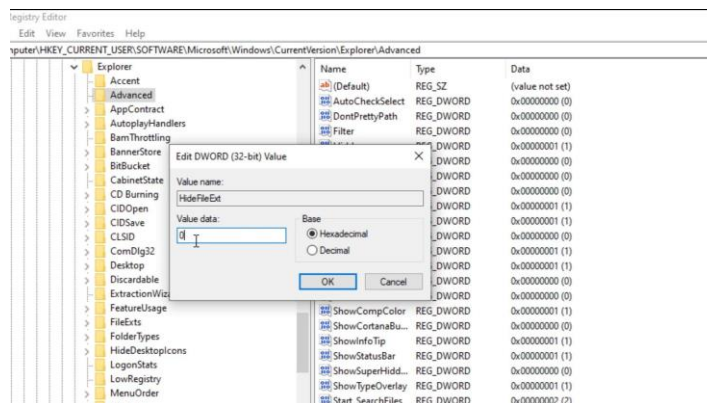
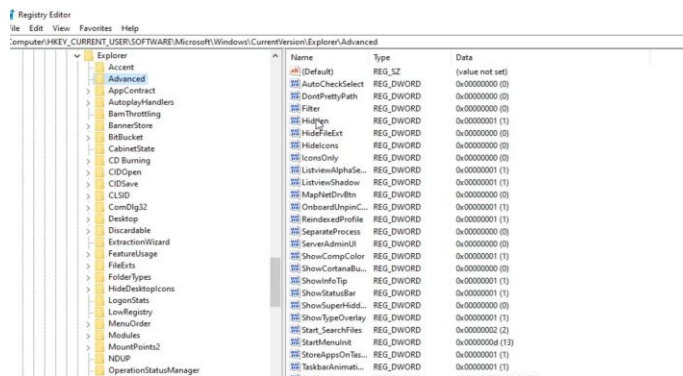
Type	Name
ALPC Port	\Windows\ApiPort
ALPC Port	\Windows\SvcApiPort
Directory	\Sessions\BNO\LINKS
Directory	\Sessions\I\
Directory	\Sessions\I\DosDevices
Directory	\Windows
Directory	\Sessions\I\AppContainerNamedObjects
Directory	\BaseNamedObjects
Directory	\BaseNamedObjects\Restricted
Directory	\BaseNamedObjects
Event	\BaseNamedObjects\EventFileExit
Event	\BaseNamedObjects\TermSrvReadyEvent
Event	\BaseNamedObjects\WinSta0\DesktopSwitch

Registry Editor

File Edit View Favorites Help

Computer\HKEY_CURRENT_USER\SOFTWARE

Name	Type
(Default)	REG_SZ



Conclusion

This practical demonstrates the differences between HTTP and HTTPS traffic by capturing and analyzing them in Wireshark. While HTTP is unencrypted and exposes sensitive information, HTTPS ensures encrypted and secure communication. It is essential to use HTTPS for secure transactions, especially for websites that involve sensitive data.

Part B: Exploring Processes, Threads, Handles, and Windows Registry

Objective

The objective of this practical is to explore and understand the working of processes, threads, handles, and the Windows Registry in an operating system. This experiment demonstrates how to monitor system processes, manage threads, view handles, and explore the Windows Registry to gain insights into system configuration and behavior.

Introduction

Processes:

A process is an instance of a running program. It is an execution context that includes program code, memory, and resources like file handles and system resources.

Threads:

A thread is a smaller unit of a process that executes a specific task within the program. Multiple threads can exist within a process, running concurrently and sharing the same resources.

Handles:

A handle is a unique identifier used by the operating system to manage resources such as files, registry keys, and memory blocks. Handles are assigned when resources are opened or allocated.

Windows Registry:

The Windows Registry is a hierarchical database that stores low-level settings and configuration data for the operating system and installed applications. It contains keys and values that define the system's behavior and settings.

Tools Used

- Task Manager: For monitoring processes and threads.
- Process Explorer: A more advanced tool for detailed process and thread information.
- Registry Editor (regedit): For exploring the Windows Registry.
- Handles (from Sysinternals): To inspect open handles for processes.

Steps

Step 1: Exploring Processes and Threads Using Task Manager

1. Open Task Manager:
 - Press Ctrl + Shift + Esc or right-click the taskbar and select Task Manager.
2. View Running Processes:
 - Go to the Processes tab to see all running processes on the system.
 - Observe columns like Name, PID (Process ID), Memory Usage, and CPU Usage.
3. Explore Threads:
 - Click on a process to highlight it.
 - Click Details (or Performance tab for more advanced CPU/thread metrics).
 - You can view threads and their CPU usage in the Threads section. Each thread has an ID and specific CPU consumption.

Step 2: Using Process Explorer for Detailed Process and Thread Information

1. Download and Open Process Explorer:
 - Download Process Explorer from Microsoft's Sysinternals website.
 - Open the application. It provides more detailed insights than Task Manager.
2. Inspect Processes and Threads:
 - View the Name and PID of each process.
 - Expand each process to view its Threads.
 - Right-click on any thread and select Properties to inspect its stack trace, priority, and other characteristics.
3. Analyze Handles:
 - In Process Explorer, you can also view Handles (objects like files, registry keys, etc.) opened by each process.
 - Select a process and click on the Handles tab to list the handles.
 - Right-click on a handle to open it or see details about it.

Step 3: Exploring the Windows Registry Using regedit

1. Open the Registry Editor:
 - Press Win + R, type regedit, and hit Enter to open the Windows Registry Editor.
2. Explore Registry Keys:
 - The Registry Editor has a hierarchical structure, with Keys and Values.
 - Browse through the main sections like HKEY_LOCAL_MACHINE, HKEY_CURRENT_USER, HKEY_CLASSES_ROOT, etc.
 - Keys are folders that store configuration data, while Values are the actual data inside those keys.
3. Find Installed Applications:
 - Go to HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall.
 - Here, you will find entries for all installed applications, with values like the installation path and version information.
4. Export and Modify Registry Keys:
 - Right-click on a key or value and select Export to save it to a .reg file.
 - You can modify values directly, but be cautious as changing registry settings can affect system behavior.

Step 4: Exploring Process Handles Using Sysinternals Handle Utility

1. Download Handle Utility:
 - Download Handle from the Sysinternals website.
 - This utility allows you to view and search for handles associated with processes.
2. List All Handles:
 - Open a Command Prompt with administrator privileges.
 - Navigate to the folder where you downloaded Handle.
 - Run the command handle.exe to list all open handles on your system.
3. Filter Handles by Process:
 - To search for handles specific to a process, use the command:
handle.exe -p <PID>

- Replace <PID> with the actual process ID.
- 4. Identify Handles:
 - The output will show you all handles (files, registry keys, memory, etc.) that are currently opened by the process.

Step 5: Analyzing System Configuration in the Windows Registry

1. Examine System Boot Configuration:
 - Navigate to
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\Configuration.
 - Here you can find boot configuration data like BootExecute and other boot settings.
2. Inspect Network Configuration:
 - Browse to
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters.
 - You will find network-related settings, including the computer's DNS, IP address, and DHCP settings.

Conclusion

This practical demonstrates the critical concepts of processes, threads, handles, and the Windows Registry. By using tools like Task Manager, Process Explorer, the Registry Editor, and Sysinternals Handle, we gain a deeper understanding of how the operating system manages resources and configuration data.

- Processes are instances of programs running on the system, and each process can contain multiple threads.
- Handles provide a way for processes to interact with system resources like files and registry keys.
- The Windows Registry stores important system configuration data, which can be accessed, modified, and exported.

By exploring and analyzing these components, you can gain valuable insights into system behavior and troubleshoot or optimize configurations effectively.

Practical No: 5

Aim: Perform a practical to Attack on a MySQL Database by using PCAP file.

Objective

The objective of this practical is to demonstrate how an attacker can exploit unencrypted MySQL traffic captured in a PCAP file to extract sensitive information, such as login credentials, and potentially compromise a MySQL database.

Introduction

MySQL is a popular open-source relational database management system. However, if MySQL traffic is not encrypted (i.e., the default configurations are used), an attacker can intercept and analyze the traffic, potentially stealing sensitive information such as usernames, passwords, and even query contents.

PCAP (Packet Capture) files store network packets and can be analyzed to extract such sensitive data. Tools like Wireshark or tcpdump are commonly used to capture network traffic and perform packet analysis.

Tools Required

1. Kali Linux (for attack and analysis).
2. Wireshark (for packet capture and analysis).
3. MySQL server (for target database).
4. PCAP file (containing MySQL traffic).
5. MySQL client (for performing attacks once credentials are acquired).

Prerequisites

1. PCAP file with MySQL traffic (captured through Wireshark or tcpdump).
2. A basic understanding of MySQL and its network traffic.
3. Wireshark installed on your system for packet capture and analysis.

Steps for the Practical

Step 1: Capture MySQL Traffic

If you do not have a pre-existing PCAP file with MySQL traffic, follow these steps to capture the traffic in your environment:

1. Install Wireshark on Kali Linux (if not installed):

```
sudo apt update
```

```
sudo apt install wireshark
```

2. Start Wireshark and begin capturing traffic:

```
sudo wireshark
```

3. Select the network interface that is actively being used.
4. Use the filter `tcp.port == 3306` to isolate MySQL traffic. This filter ensures only traffic on the MySQL port (3306) is shown.

Wireshark filter:

```
tcp.port == 3306
```

5. Start the capture, initiate some MySQL traffic (such as connecting to the database or performing queries), and stop the capture once enough packets have been recorded.
6. Save the captured traffic as a .pcap file for further analysis.

Step 2: Analyze the PCAP File

1. Open Wireshark and load the saved PCAP file.
2. Use the filter `mysql or tcp.port == 3306` to focus on MySQL traffic.

Wireshark filter:

`tcp.port == 3306`

3. Look for cleartext passwords or other sensitive data in the packets.
 - MySQL often transmits usernames and passwords in plaintext if no encryption is enabled (default behavior).
 - You can find these credentials in the initial connection packets between the MySQL client and server.
4. Inspect packet details:
 - Look for packets labeled as "Login Request" or "Query" that contain unencrypted data.
 - The login packet will often contain the username and password in plaintext.
5. Extract Credentials:
 - Once you identify the packet containing the login credentials, extract the username and password from the packet details.

Step 3: Use the Credentials to Perform an Attack

Now that you have the credentials (username and password) from the PCAP file, you can perform an attack on the MySQL database:

1. Login to the MySQL database: Use the extracted credentials to attempt login into the MySQL server from the command line:
`mysql -u <username> -p<password> -h <target_ip>`
2. Access Database Information: Once logged in, you can attempt to enumerate databases, tables, and users to gain further access:

`SHOW DATABASES;`

`USE <database_name>;`

`SHOW TABLES;`

3. Extract Sensitive Data: Query the databases and extract any valuable or sensitive information:
`SELECT * FROM <table_name>;`

Step 4: Mitigate the Attack

After demonstrating the attack, it's important to address the security vulnerabilities in MySQL:

1. Enable SSL/TLS Encryption:
 - MySQL supports encrypted connections using SSL/TLS, which prevents the transmission of sensitive data in plaintext.
 - To enable SSL/TLS, configure the MySQL server to use SSL certificates.
2. Use Strong Authentication:
 - Implement strong password policies to make it harder for attackers to guess credentials.
 - Enable multi-factor authentication (MFA) for MySQL access if possible.
3. Restrict Network Access:
 - Limit the MySQL server's access to trusted IP addresses only.

- Use firewall rules to prevent unauthorized access to MySQL ports (3306).
- 4. Encrypt Data at Rest:
 - Use encryption mechanisms to protect sensitive data stored in MySQL databases.

Conclusion

This practical demonstrates how an attacker can capture MySQL traffic in a PCAP file and exploit unencrypted login credentials to gain unauthorized access to a MySQL database. The capture and analysis of network traffic can reveal sensitive information that can be used for further attacks. Therefore, it is essential to ensure MySQL traffic is encrypted to protect sensitive data from being exposed to attackers.

Practical No: 6

Aim: Create your own syslog Server

Objective

The objective of this practical is to set up and configure a syslog server using rsyslog on a Linux machine. We will configure a remote syslog server to collect log messages from multiple client machines, allowing for centralized logging and easier monitoring.

Introduction

A syslog server is a system used to collect log messages sent by various network devices and services (e.g., servers, routers, firewalls). By using a syslog server, you can centralize your logging system, making it easier to monitor events, troubleshoot, and analyze system behavior.

In this practical, we will set up a basic syslog server that collects logs from remote clients, allowing you to monitor system activities in one place.

Tools Required

1. Linux machine (Ubuntu, Kali Linux, CentOS, etc.) as the syslog server.
2. rsyslog (default syslog service on Linux).
3. Syslog clients (other Linux machines or network devices that send logs).

Prerequisites

1. Basic understanding of syslog and logging systems.
2. Sudo or root access on the Linux server.
3. Basic knowledge of network configurations.
4. rsyslog installed on your system (it is usually installed by default on most Linux distributions).

Steps for the Practical

Step 1: Install and Configure rsyslog on the Syslog Server

1. Install rsyslog (if not installed already): On most Linux distributions, rsyslog comes pre-installed. However, if it is not installed, use the following command to install it.

On Ubuntu/Debian:

```
sudo apt update
```

```
sudo apt install rsyslog
```

On CentOS/RHEL:

```
sudo yum install rsyslog
```

2. Enable and start the rsyslog service: Ensure that the rsyslog service is enabled and started.

```
sudo systemctl enable rsyslog
```

```
sudo systemctl start rsyslog
```

3. Configure rsyslog to accept remote logs: By default, rsyslog only listens for local logs. To configure it to accept remote logs, you need to modify its configuration.

Open the rsyslog configuration file:

```
sudo nano /etc/rsyslog.conf
```

4. Uncomment or add the following lines to enable the server to receive remote syslog messages:

```
# Provides UDP syslog reception
$ModLoad imudp
$UDPServerRun 514
```

```
# Provides TCP syslog reception
$ModLoad imtcp
$InputTCPServerRun 514
```

- 514 is the default port for syslog messages.
- The imudp and imtcp modules allow the server to listen for UDP and TCP syslog messages, respectively.

5. Restart the rsyslog service to apply the changes:

```
sudo systemctl restart rsyslog
```

Step 2: Verify rsyslog is Listening for Remote Log Messages

1. Check if rsyslog is listening on port 514: Use the netstat or ss command to check if rsyslog is listening on UDP/TCP port 514.

```
sudo netstat -tuln | grep 514
```

You should see output indicating that rsyslog is listening on port 514.

Step 3: Configure Syslog Clients to Send Logs to the Syslog Server

On the client machines (the systems from which you want to collect logs), you need to configure them to send syslog messages to the server.

1. Edit the rsyslog configuration file on the client machine:

```
sudo nano /etc/rsyslog.conf
```

2. Add the following line at the end of the file to forward logs to the syslog server:

markdown

```
@<syslog_server_ip>:514
```

server.

- Use @ for UDP (unreliable) or @@ for TCP (more reliable).
- Example for UDP:

```
@192.168.1.10:514
```

- Example for TCP:

```
@192.168.1.10:514
```

3. Restart rsyslog on the client to apply the configuration:

```
sudo systemctl restart rsyslog
```

Step 4: Test Syslog Communication

1. On the client machine, generate a log message to verify that it is being sent to the server:

```
logger "This is a test message from the client"
```

2. On the syslog server, check the logs to see if the message from the client has been received:

Check the /var/log/syslog or /var/log/messages file, depending on your Linux distribution:

```
sudo tail -f /var/log/syslog
```

You should see the log message sent by the client.

Step 5: Optional - Configure Log Rotation (for Syslog Files)

Log files can grow quickly, especially when collecting logs from multiple systems. To avoid filling up the disk, you should configure log rotation.

1. Open the log rotation configuration file:
`sudo nano /etc/logrotate.d/rsyslog`
2. Set the parameters for log rotation, such as frequency and maximum number of logs to keep. For example:

```
/var/log/syslog {
    weekly
    rotate 4
    compress
    delaycompress
    notifempty
    missingok
}
```

- weekly: Rotate logs weekly.
- rotate 4: Keep 4 archived logs.
- compress: Compress old logs.
- notifempty: Do not rotate empty log files.

Step 6: Secure Your Syslog Server (Optional)

For better security, consider using TLS encryption for syslog traffic:

1. Set up SSL certificates on both the server and client to secure the syslog traffic.
2. Configure rsyslog to use SSL by editing the configuration files to enable TLS and specifying the paths to the certificate files.

```
→ sudo apt install rsyslog
rsyslog is already the newest version (8.2410.0-1).
The following package was automatically installed and is no longer required:
tcpd
Use 'sudo apt autoremove' to remove it.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1790

(kali@kali)-[~]
$ sudo systemctl enable rsyslog
(kali@kali)-[~]
$ sudo systemctl start rsyslog
(kali@kali)-[~]
$ sudo nano /etc/rsyslog.conf
(kali@kali)-[~]
$ sudo systemctl restart rsyslog
```

```
/etc/rsyslog.conf: configuration file for rsyslog
#
# For more information install rsyslog-doc and see
# /usr/share/doc/rsyslog-doc/html/configuration/index.html

#####
#### MODULES ####
#####

module(load="imuxsock") # provides support for local system logging
module(load="imklog")   # provides kernel logging support
#module(load="immark")  # provides --MARK-- message capability

# provides UDP syslog reception
#module(load="imudp")
#input(type="imudp" port="514")

# provides TCP syslog reception
#module(load="imtcp")
#input(type="imtcp" port="514")

[ Read 69 lines ]
Help      Write Out  Where Is  Cut       Execute   Location  M-U Undo
Exit      Read File  Replace   Paste     Justify   Go To Line M-E Redo
```

```
kali@kali: ~  
File Actions Edit View Help  
[kali@kali]~  
$ sudo apt update  
[sudo] password for kali:  
Get:1 http://kali.download/kali kali-rolling InRelease [41.5 kB]  
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.3 MB]  
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [49.4 MB]  
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [112 kB]  
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [274 kB]  
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [197 kB]  
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [876 kB]  
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [10.6 kB]  
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [23.1 kB]  
Fetched 71.2 MB in 41s (1,754 kB/s)  
1790 packages can be upgraded. Run 'apt list --upgradable' to see them.  
[kali@kali]~  
$
```

```
#####  
### MODULES ###  
#####  
  
module(load="imuxsock") # provides support for local system login  
module(load="imklog") # provides kernel logging support  
#module(load="immark") # provides --MARK-- message capability  
  
# provides UDP syslog reception  
#module(load="imudp")  
#input(type="imudp" port="514")  
  
# provides TCP syslog reception  
#module(load="imtcp")  
#input(type="imtcp" port="514")
```

```
[kali@kali]~  
$ sudo systemctl start rsyslog  
[kali@kali]~  
$ sudo nano /etc/rsyslog.conf  
[kali@kali]~  
$ sudo systemctl restart rsyslog  
[kali@kali]~  
$ sudo ufw allow 514/tcp  
Skipping adding existing rule  
Skipping adding existing rule (v6)  
[kali@kali]~  
$ sudo ufw allow 514/udp  
Skipping adding existing rule  
Skipping adding existing rule (v6)  
[kali@kali]~  
$ logger -n 127.0.0.1 P 514 "Test for ismile"
```

```
(kali@kali)-[~]
$ logger -n 127.0.0.1 P 514 "Test for ismile"

(kali@kali)-[~]
$ tail -f /var/log/syslog
2024-11-16T17:45:01.825468+05:30 kali CRON[3828]: (root) CMD (command -v del
an-sa1 1 1)
2024-11-16T17:45:46.211470+05:30 kali systemd[1]: Stopping rsyslog.service :
2024-11-16T17:45:46.212189+05:30 kali rsyslogd: [origin software="rsyslogd"
"525" x-info="https://www.rsyslog.com"] exiting on signal 15.
2024-11-16T17:45:46.212368+05:30 kali systemd[1]: rsyslog.service: Deactiva
2024-11-16T17:45:46.212571+05:30 kali systemd[1]: Stopped rsyslog.service -
2024-11-16T17:45:46.234028+05:30 kali systemd[1]: Starting rsyslog.service :
2024-11-16T17:45:46.432138+05:30 kali rsyslogd: imuxsock: Acquired UNIX soci
slog' (fd 3) from systemd. [v8.2410.0]
2024-11-16T17:45:46.432483+05:30 kali systemd[1]: Started rsyslog.service -
2024-11-16T17:45:46.432638+05:30 kali rsyslogd: [origin software="rsyslogd"
"4201" x-info="https://www.rsyslog.com"] start
2024-11-16T17:47:43.692243+05:30 kali kali P 514 Test for ismile
```

Conclusion

In this practical, you have successfully set up a syslog server on a Linux machine using rsyslog. You configured the server to accept remote log messages from client machines and verified the communication between the client and server by generating test log messages. You also learned how to rotate logs and can optionally secure your syslog communication with encryption for a more secure setup.

This centralized logging approach allows administrators to monitor and analyze logs from multiple systems in one place.

Practical No: 7

Aim: Configure your Linux system to send syslog messages to a syslog server and Read them.

Objective

The objective of this practical is to configure a Linux system to send Syslog messages to a remote Syslog server and analyze the received logs. This setup enables centralized log management, which enhances security monitoring and troubleshooting.

Introduction

Syslog and Its Importance

Syslog is a standard protocol used to log system messages from different devices and applications in a centralized location. It helps administrators track system events, detect anomalies, and manage security incidents.

Client-Server Architecture of Syslog

- A Syslog Server collects and stores logs from various clients.
- A Syslog Client sends logs to the configured Syslog server over UDP (default) or TCP.

In this practical, we will:

1. Set up a Syslog server.
2. Configure a Syslog client to send logs to the server.
3. Verify the logs received on the server.

Commands Used

Step 1: Setting Up Virtual Machines

- Use Oracle VirtualBox 7.0 to create two Kali Linux virtual machines:
 - One as the Syslog Server.
 - One as the Syslog Client.
- If needed, clone an existing VM to create the second machine.

Checking Network Configuration

Find the IP addresses of both machines:

ifconfig

Example output:

- Syslog Server IP: 192.168.0.136
- Syslog Client IP: 192.168.0.244

Ensure network connectivity:

ping 192.168.0.136

Step 2: Configuring the Syslog Server

1. Update the system

sudo apt update

2. Install rsyslog (if not installed)

sudo apt install rsyslog -y

3. Enable and start the Syslog service

sudo systemctl enable rsyslog

sudo systemctl start rsyslog

4. Verify that rsyslog is running
`sudo systemctl status rsyslog`
5. Modify the Syslog configuration to accept remote logs
`sudo nano /etc/rsyslog.conf`
 - Locate and uncomment the following lines to allow UDP and TCP connections:
`$ModLoad imudp`
`$UDPServerRun 514`

`$ModLoad imtcp`
`$InputTCPServerRun 514`
 - Save and exit (CTRL+X → Y → ENTER).
6. Restart the rsyslog service
`sudo systemctl restart rsyslog`

Step 3: Configuring the Syslog Client

1. Update the system
`sudo apt update`
2. Install rsyslog
`sudo apt install rsyslog -y`
3. Modify the Syslog configuration file
`sudo nano /etc/rsyslog.conf`
 - At the end of the file, add the following lines to forward logs to the server:
`graphql`
`*.info @192.168.0.136:514`
`*.info @@192.168.0.136:514`
 - Save and exit (CTRL+X → Y → ENTER).
4. Restart the rsyslog service

`sudo systemctl restart rsyslog`

Step 4: Testing and Verifying Logs

On the Syslog Server

To monitor incoming logs in real time:

```
sudo tail -f /var/log/syslog
```

On the Syslog Client

Send a test message to the Syslog server:

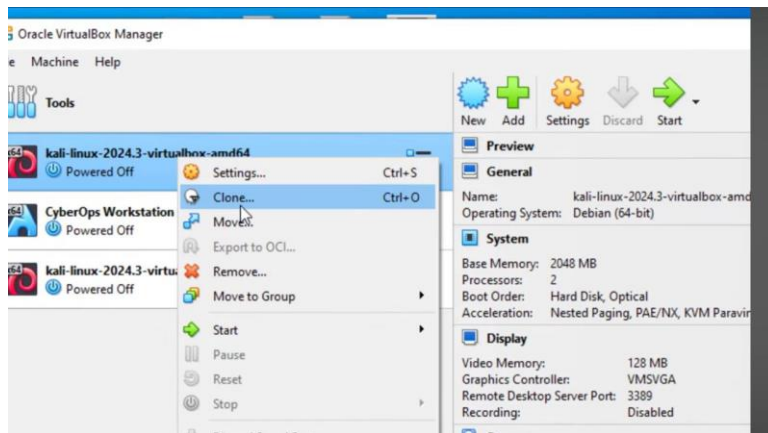
```
logger -n 192.168.0.136 -P 514 "Test message from Syslog client"
```

Verify the Log on the Server

Check if the message appears:

```
sudo tail -f /var/log/syslog
```

If successful, the message should appear in the logs.



```
kali@kali: ~
File Actions Edit View Help
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.30.136 netmask 255.255.255.0 broadcast 192.168.30.255
    inet6 fe80::c8f6:1c2:a90c:8fcd prefixlen 64 scopeid 0<link>
    inet6 2402:3a80:4171:70a1:4ae0:a262:cbfe:5297 prefixlen 64 scopeid 0<global>
    ether 08:00:27:ad:25:87 txqueuelen 1000 (Ethernet)
    RX packets 16 bytes 2510 (2.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 29 bytes 4791 (4.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<lo>host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@kali)~$
```

```
kali@kali: ~
File Actions Edit View Help
└─(kali@kali)~$ sudo apt update
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
1810 packages can be upgraded. Run 'apt list --upgradable' to see them.

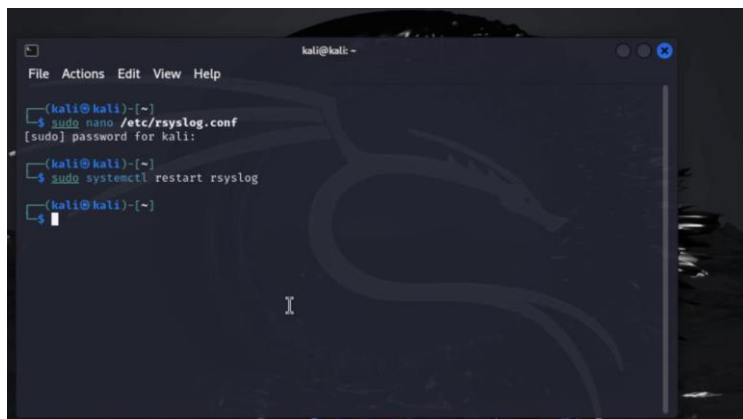
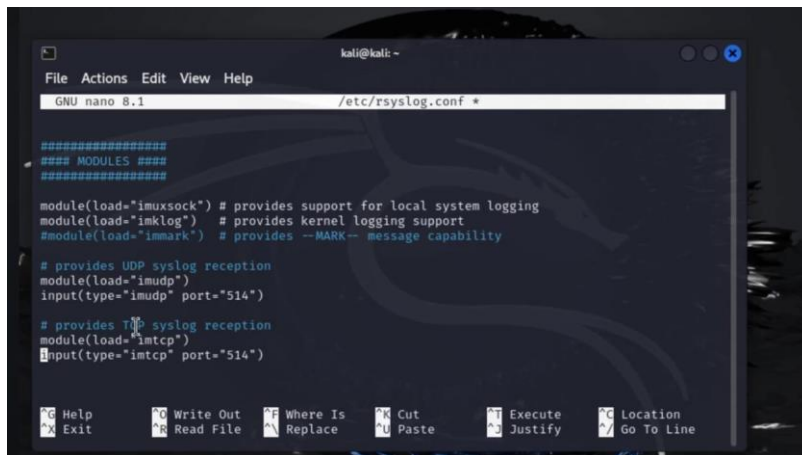
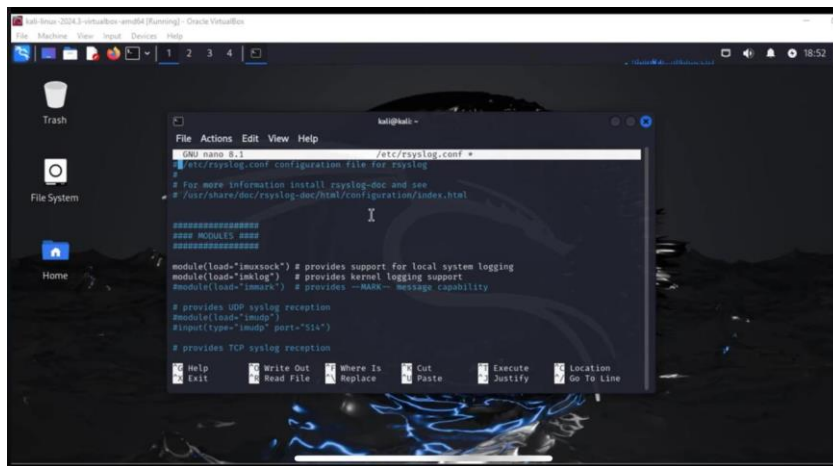
(kali@kali)~$ sudo apt install rsync
```

```
kali@kali: ~  
File Actions Edit View Help  
1810 packages can be upgraded. Run 'apt list --upgradable' to see them.  
  
(kali@kali)~  
$ sudo apt install rsyslog  
rsyslog is already the newest version (8.2410.0-1).  
The following packages were automatically installed and are no longer required:  
dnsmap python3-bottleneck python3-psychodan rwhod  
figlet python3-cpuinfo python3-qasync smtp-user-enum  
finger python3-git python3-serial-asyncio sparta-scripts  
imagemagick python3-gitdb python3-smmmap tcpd  
imagemagick-6.q16 python3-numexpr python3-tables toilet-fonts  
libblosc2-3 python3-odf python3-tables-lib unicornscan  
medusa python3-pandas python3-tld urlscan  
python-odf-doc python3-pandas-lib python3-yaswfp wapiti  
python-odf-tools python3-pyexploitdb rsh-redone-client  
python-tables-data python3-pyfiglet rwho  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1810  
  
(kali@kali)~  
$
```

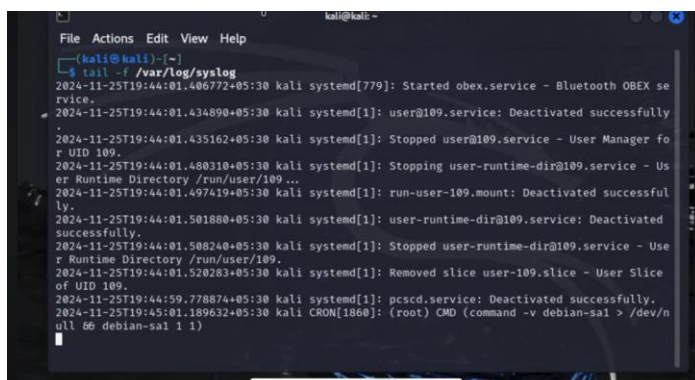
```
kali@kali: ~  
File Actions Edit View Help  
dnsmap python3-bottleneck python3-psychodan rwhod  
figlet python3-cpuinfo python3-qasync smtp-user-enum  
finger python3-git python3-serial-asyncio sparta-scripts  
imagemagick python3-gitdb python3-smmmap tcpd  
imagemagick-6.q16 python3-numexpr python3-tables toilet-fonts  
libblosc2-3 python3-odf python3-tables-lib unicornscan  
medusa python3-pandas python3-tld urlscan  
python-odf-doc python3-pandas-lib python3-yaswfp wapiti  
python-odf-tools python3-pyexploitdb rsh-redone-client  
python-tables-data python3-pyfiglet rwho  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1810  
  
(kali@kali)~  
$ sudo systemctl enable rsyslog  
  
(kali@kali)~  
$ sudo systemctl start rsyslog
```

```
kali@kali: ~  
File Actions Edit View Help  
  
(kali@kali)~  
$ sudo systemctl status rsyslog  
● rsyslog.service - System Logging Service  
Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)  
Active: active (running) since Mon 2024-11-25 17:47:40 IST; 49min ago  
Invocation: 9afe0c7beb7e48a2b425fa8d432ecb3f  
TriggeredBy: ● syslog.socket  
Docs: man:rsyslogd(8)  
man:rsyslog.conf(5)  
https://www.rsyslog.com/doc/  
Main PID: 499 (rsyslogd)  
Tasks: 10 (limit: 2268)  
Memory: 3.9M (peak: 4.2M)  
CPU: 311ms  
CGroup: /system.slice/rsyslog.service  
└─499 /usr/sbin/rsyslogd -n -iNONE  
  
Nov 25 17:47:39 kali systemd[1]: Starting rsyslog.service - System Logging Service...  
Nov 25 17:47:40 kali rsyslogd[499]: imuxsock: Acquired UNIX socket '/run/systemd/journal/sy  
Nov 25 17:47:40 kali systemd[1]: Started rsyslog.service - System Logging Service.  
Nov 25 17:47:40 kali rsyslogd[499]: [origin software="rsyslogd" swVersion="8.2410.0" x-pid=]  
lines 1-19/19 (END)
```

```
kali@kali: ~  
File Actions Edit View Help  
● rsyslog.service - System Logging Service  
Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)  
Active: active (running) since Mon 2024-11-25 17:47:40 IST; 49min ago  
Invocation: 9afe0c7beb7e48a2b425fa8d432ecb3f  
TriggeredBy: ● syslog.socket  
Docs: man:rsyslogd(8)  
man:rsyslog.conf(5)  
https://www.rsyslog.com/doc/  
Main PID: 499 (rsyslogd)  
Tasks: 10 (limit: 2268)  
Memory: 3.9M (peak: 4.2M)  
CPU: 311ms  
CGroup: /system.slice/rsyslog.service  
└─499 /usr/sbin/rsyslogd -n -iNONE  
  
Nov 25 17:47:39 kali systemd[1]: Starting rsyslog.service - System Logging Service...  
Nov 25 17:47:40 kali rsyslogd[499]: imuxsock: Acquired UNIX socket '/run/systemd/journal/sy  
Nov 25 17:47:40 kali systemd[1]: Started rsyslog.service - System Logging Service.  
Nov 25 17:47:40 kali rsyslogd[499]: [origin software="rsyslogd" swVersion="8.2410.0" x-pid=]  
  
(kali@kali)~  
$ sudo nano /etc/rsyslog.conf
```



AFTER CLONE



AFTER LOGGING INFORMATION GO TO CLONE TO CHECK LOGGER

```
kali@kali: ~  
File Actions Edit View Help  
2024-11-25T19:44:01.406772+05:30 kali systemd[779]: Started obex.service - Bluetooth OBEX service.  
2024-11-25T19:44:01.434890+05:30 kali systemd[1]: user@109.service: Deactivated successfully.  
2024-11-25T19:44:01.435162+05:30 kali systemd[1]: Stopped user@109.service - User Manager for UID 109.  
2024-11-25T19:44:01.480310+05:30 kali systemd[1]: Stopping user-runtime-dir@109.service - User Runtime Directory /run/user/109...  
2024-11-25T19:44:01.497419+05:30 kali systemd[1]: run-user-109.mount: Deactivated successfully.  
2024-11-25T19:44:01.501880+05:30 kali systemd[1]: user-runtime-dir@109.service: Deactivated successfully.  
2024-11-25T19:44:01.508240+05:30 kali systemd[1]: Stopped user-runtime-dir@109.service - User Runtime Directory /run/user/109.  
2024-11-25T19:44:01.520283+05:30 kali systemd[1]: Removed slice user-109.slice - User Slice of UID 109.  
2024-11-25T19:44:59.778874+05:30 kali systemd[1]: pcsd.service: Deactivated successfully.  
2024-11-25T19:45:01.189632+05:30 kali CRON[1860]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)  
2024-11-25T19:55:01.234660+05:30 kali CRON[6743]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)  
2024-11-25T19:57:51.906978+05:30 kali Test for Ismile
```

CLONE

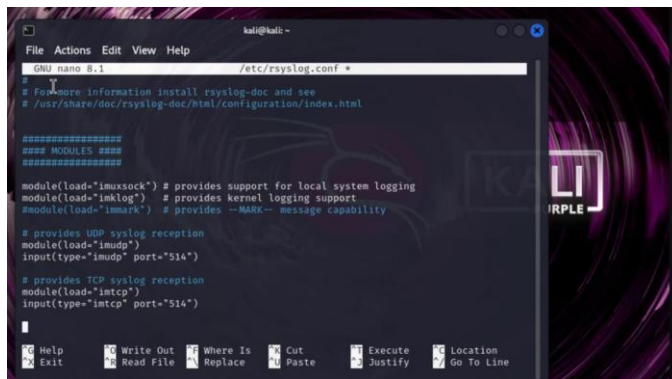
```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.30.244 netmask 255.255.255.0 broadcast 192.168.30.255  
    inet6 fe80::f87b:551d:e990:fe9f prefixlen 64 scopeid 0<link>  
    inet6 2402:3a80:4171:70a1:3973:3063:9e9d:5a66 prefixlen 64 scopeid  
l>  
    ether 08:00:27:be:12:f5 txqueuelen 1000 (Ethernet)  
    RX packets 18 bytes 3176 (3.1 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 27 bytes 4125 (4.0 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0<host>  
    loop txqueuelen 1000 (Local Loopback)
```

```
Trash  
System  
Home  
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.30.244 netmask 255.255.255.0 broadcast 192.168.30.255  
    inet6 fe80::f87b:551d:e990:fe9f prefixlen 64 scopeid 0<link>  
    inet6 2402:3a80:4171:70a1:3973:3063:9e9d:5a66 prefixlen 64 scopeid 0<global>  
l>  
    ether 08:00:27:be:12:f5 txqueuelen 1000 (Ethernet)  
    RX packets 18 bytes 3176 (3.1 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 27 bytes 4125 (4.0 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 8 bytes 480 (480.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 8 bytes 480 (480.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
(kali@kali)-[~]  
$ ping 192.168.30.136
```

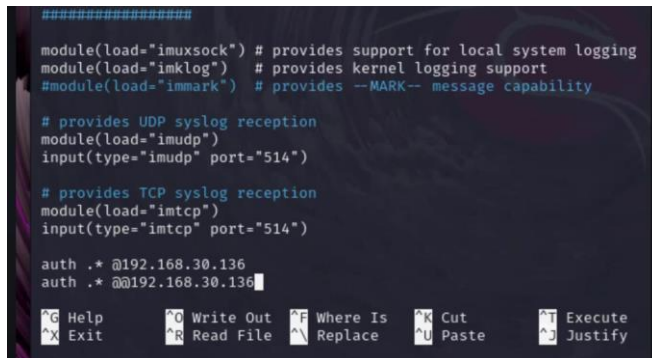
```
kali@kali: ~  
File Actions Edit View Help  
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
inet 127.0.0.1 netmask 255.0.0.0  
inet6 ::1 prefixlen 128 scopeid 0<host>  
loop txqueuelen 1000 (Local Loopback)  
RX packets 8 bytes 480 (480.0 B)  
RX errors 0 dropped 0 overruns 0 frame 0  
TX packets 8 bytes 480 (480.0 B)  
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
kali@kali:~$ ping 192.168.30.136  
PING 192.168.30.136 (192.168.30.136) 56(84) bytes of data:  
64 bytes from 192.168.30.136: icmp_seq=1 ttl=64 time=0.700 ms  
64 bytes from 192.168.30.136: icmp_seq=2 ttl=64 time=0.491 ms  
64 bytes from 192.168.30.136: icmp_seq=3 ttl=64 time=0.509 ms  
64 bytes from 192.168.30.136: icmp_seq=4 ttl=64 time=0.582 ms  
^C  
--- 192.168.30.136 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3058ms  
rtt min/avg/max/mdev = 0.491/0.570/0.700/0.082 ms  
  
kali@kali:~$
```

```
kali@kali: ~  
File Actions Edit View Help  
GNU nano 8.1 /etc/rsyslog.conf *  
For more information install rsyslog-doc and see  
# /usr/share/doc/rsyslog-doc/html/configuration/index.html  
  
#####  
### MODULES ###  
#####  
  
module(load="imuxsock") # provides support for local system logging  
module(load="imklog") # provides kernel logging support  
#module(load="immark") # provides --MARK-- message capability  
  
# provides UDP syslog reception  
#module(load="imudp")  
#input(type="imudp" port="514")  
  
# provides TCP syslog reception  
#module(load="imtcp")  
#input(type="imtcp" port="514")  
  
[H] Help [O] Write Out [F] Where Is [X] Cut [E] Execute [G] Location  
[X] Exit [R] Read File [A] Replace [U] Paste [J] Justify [/_] Go To Line
```

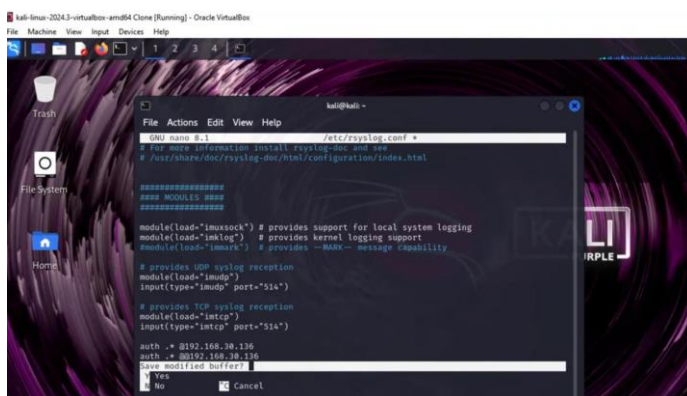
```
kali@kali: ~  
File Actions Edit View Help  
  
kali@kali:~$ sudo apt update  
[sudo] password for kali:  
Hit:1 http://http.kali.org/kali kali-rolling InRelease  
1812 packages can be upgraded. Run 'apt list --upgradable' to see them.  
  
kali@kali:~$ sudo apt install rsyslog  
rsyslog is already the newest version (8.2410.0-1).  
The following package was automatically installed and is no longer required:  
tcpd  
Use 'sudo apt autoremove' to remove it.  
  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1812  
  
kali@kali:~$ sudo nano /etc/rsyslog.conf
```



```
kali@kali: ~  
File Actions Edit View Help  
GNU nano 8.1 /etc/rsyslog.conf  
#  
# For more information install rsyslog-doc and see  
# /usr/share/doc/rsyslog-doc/html/configuration/index.html  
  
#####  
### MODULES ###  
#####  
  
module(load="imuxsock") # provides support for local system logging  
module(load="imklog") # provides kernel logging support  
#module(load="immark") # provides --MARK-- message capability  
  
# provides UDP syslog reception  
module(load="imudp")  
input(type="imudp" port="514")  
  
# provides TCP syslog reception  
module(load="imtcp")  
input(type="imtcp" port="514")  
  
#  
#
```



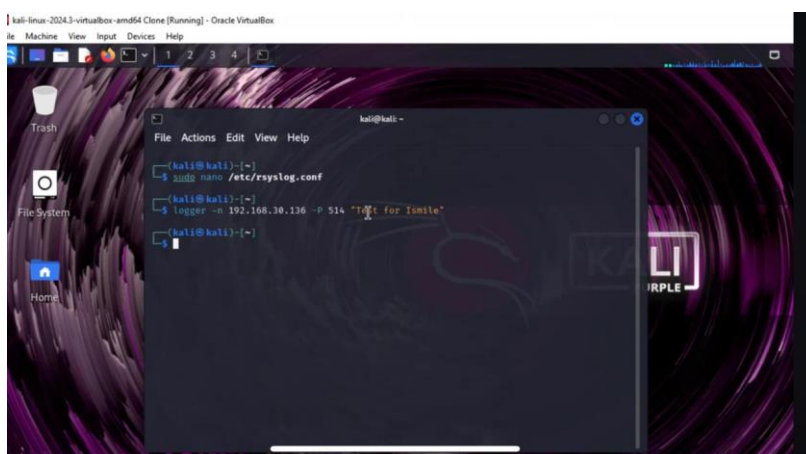
```
#####  
  
module(load="imuxsock") # provides support for local system logging  
module(load="imklog") # provides kernel logging support  
#module(load="immark") # provides --MARK-- message capability  
  
# provides UDP syslog reception  
module(load="imudp")  
input(type="imudp" port="514")  
  
# provides TCP syslog reception  
module(load="imtcp")  
input(type="imtcp" port="514")  
  
auth.* @192.168.30.136  
auth.* @@192.168.30.136  
  
#  
#
```



```
kali@kali: ~  
File Actions Edit View Help  
GNU nano 8.1 /etc/rsyslog.conf  
#  
# For more information install rsyslog-doc and see  
# /usr/share/doc/rsyslog-doc/html/configuration/index.html  
  
#####  
### MODULES ###  
#####  
  
module(load="imuxsock") # provides support for local system logging  
module(load="imklog") # provides kernel logging support  
#module(load="immark") # provides --MARK-- message capability  
  
# provides UDP syslog reception  
module(load="imudp")  
input(type="imudp" port="514")  
  
# provides TCP syslog reception  
module(load="imtcp")  
input(type="imtcp" port="514")  
  
#  
#
```

GO BACK TO SERVER

AFTER LOGGING INFORMATION CHECK LOGGER IN CLONE



```
kali@kali: ~  
File Actions Edit View Help  
GNU nano 8.1 /etc/rsyslog.conf  
#  
# For more information install rsyslog-doc and see  
# /usr/share/doc/rsyslog-doc/html/configuration/index.html  
  
#####  
### MODULES ###  
#####  
  
module(load="imuxsock") # provides support for local system logging  
module(load="imklog") # provides kernel logging support  
#module(load="immark") # provides --MARK-- message capability  
  
# provides UDP syslog reception  
module(load="imudp")  
input(type="imudp" port="514")  
  
# provides TCP syslog reception  
module(load="imtcp")  
input(type="imtcp" port="514")  
  
#  
#
```

AFTER LOGGER GO TO SERVER TO CHECK MESSAGE.

Observations

1. Successful log transmission: Messages sent from the Syslog Client appear on the Syslog Server.
2. Centralized logging: All logs from multiple devices can be stored in one place for analysis.
3. Use of TCP vs. UDP:
 - UDP (@): Faster but may drop logs if there is network congestion.
 - TCP (@@): More reliable as it ensures message delivery.

Conclusion

This practical demonstrates how to set up centralized logging using rsyslog. By configuring a Syslog Server and Syslog Client, we successfully sent system logs over the network. Centralized logging is crucial for monitoring security events, troubleshooting system issues, and maintaining compliance in enterprise environments.

Practical No: 8

Aim: Install and Run Splunk on Linux

Objective

The objective of this practical is to install and configure Splunk on a Linux system. Splunk is a powerful data analysis and monitoring tool used for searching, monitoring, and analyzing machine-generated big data via a web-style interface.

Introduction

Splunk is a platform for searching, monitoring, and analyzing machine data, mainly for troubleshooting, security monitoring, and operational intelligence. This practical demonstrates the installation and basic setup of Splunk on a Linux machine.

Tools Used

- Linux system (Ubuntu, CentOS, or similar)
- Splunk Installation Package (available on the Splunk website)
- Terminal/Command Line Interface

Prerequisites

- A Linux system (e.g., Ubuntu, CentOS).
- Sudo/root privileges to install software.
- Access to the internet for downloading Splunk.
- A basic understanding of Linux terminal commands.

Steps for Installing and Running Splunk on Linux

Step 1: Download the Splunk Installation Package

1. Visit the Splunk download page from a browser.
2. Choose the appropriate Splunk version for Linux.
 - For example, for 64-bit Linux, download the .tgz file.

Alternatively, you can use `wget` to download directly to your Linux machine:

```
wget -O splunk-<version>-Linux-x86_64.tgz 'https://download.splunk.com/path-to-splunk-version.tgz'
```

Replace `<version>` with the appropriate version of Splunk.

Step 2: Install Splunk

1. Open the terminal on your Linux system.
2. Navigate to the directory where the Splunk package is downloaded:
`cd /path/to/download`
3. Extract the .tgz file using the tar command:
`tar -xvzf splunk-<version>-Linux-x86_64.tgz`
4. Move the extracted Splunk folder to the desired directory, e.g., `/opt`:
`sudo mv splunk-<version> /opt/splunk`

Step 3: Set Up Permissions

1. Set proper permissions to allow the Splunk service to run correctly:
`sudo chown -R splunk:splunk /opt/splunk`
2. Grant the necessary permissions to allow the splunk user to read and write logs:
`sudo chmod -R 777 /opt/splunk`

Step 4: Start Splunk

1. Change to the Splunk directory:
`cd /opt/splunk/bin`
2. Start Splunk:
`sudo ./splunk start --accept-license`
 - The `--accept-license` flag automatically accepts the Splunk license.
 - When you run this command for the first time, Splunk will prompt you to accept the license agreement.
3. Create a password for the Splunk admin account when prompted.

Step 5: Access the Splunk Web Interface

1. After Splunk starts, open your web browser and navigate to:
`http://<your-server-ip>:8000`
 - Replace `<your-server-ip>` with the actual IP address of the Linux server.
2. Login using the admin username and the password you created during the installation.

Step 6: Configure Splunk

1. Once logged in, you will see the Splunk dashboard.
2. Follow the on-screen instructions to configure Splunk, add data sources, and set up monitoring.

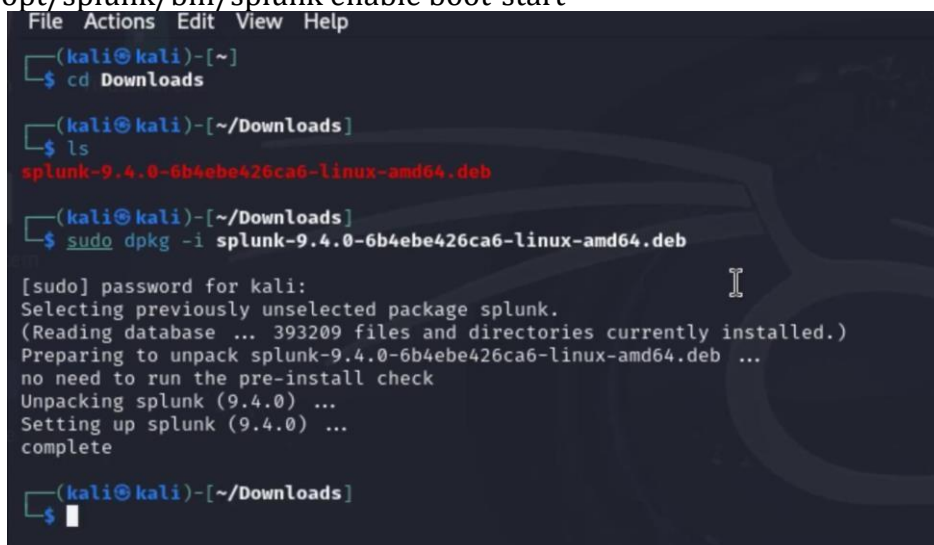
Step 7: Verify the Installation

1. On the Splunk dashboard, check for logs, data inputs, and system health.
2. If Splunk is running properly, the "Search & Reporting" option should show no errors.

Step 8: Start Splunk on Boot (Optional)

If you want Splunk to start automatically upon system boot, run the following command:

`sudo /opt/splunk/bin/splunk enable boot-start`



```
File Actions Edit View Help
(kali@kali)-[~]
$ cd Downloads
(kali@kali)-[~/Downloads]
$ ls
splunk-9.4.0-6b4ebe426ca6-linux-amd64.deb
(kali@kali)-[~/Downloads]
$ sudo dpkg -i splunk-9.4.0-6b4ebe426ca6-linux-amd64.deb
[sudo] password for kali:
Selecting previously unselected package splunk.
(Reading database ... 393209 files and directories currently installed.)
Preparing to unpack splunk-9.4.0-6b4ebe426ca6-linux-amd64.deb ...
no need to run the pre-install check
Unpacking splunk (9.4.0) ...
Setting up splunk (9.4.0) ...
complete
(kali@kali)-[~/Downloads]
$
```

```
ls
microsoft splunk

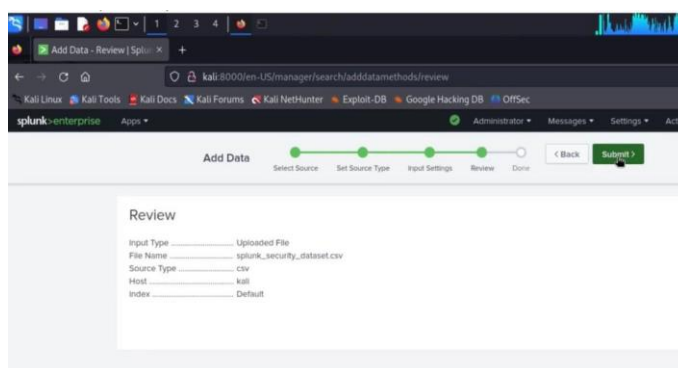
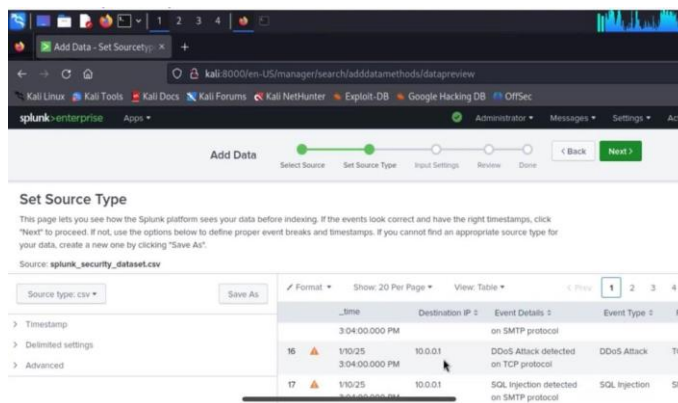
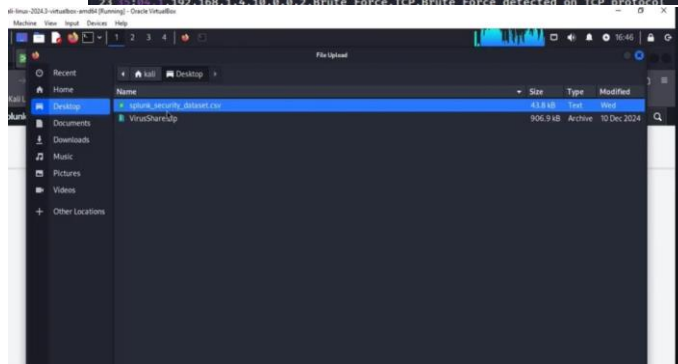
(kali@kali)-[/opt]
$ cd splunk

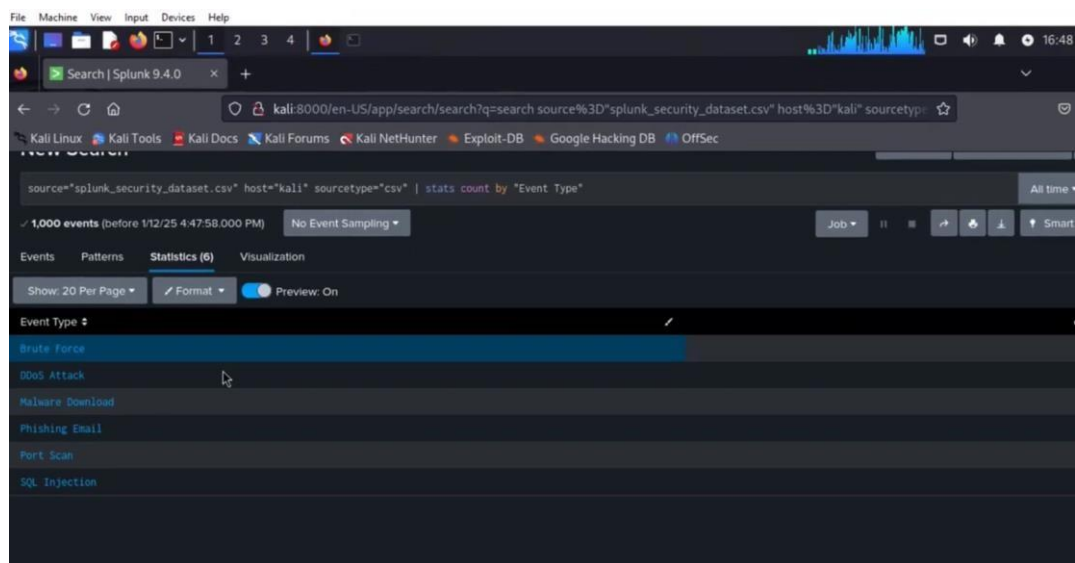
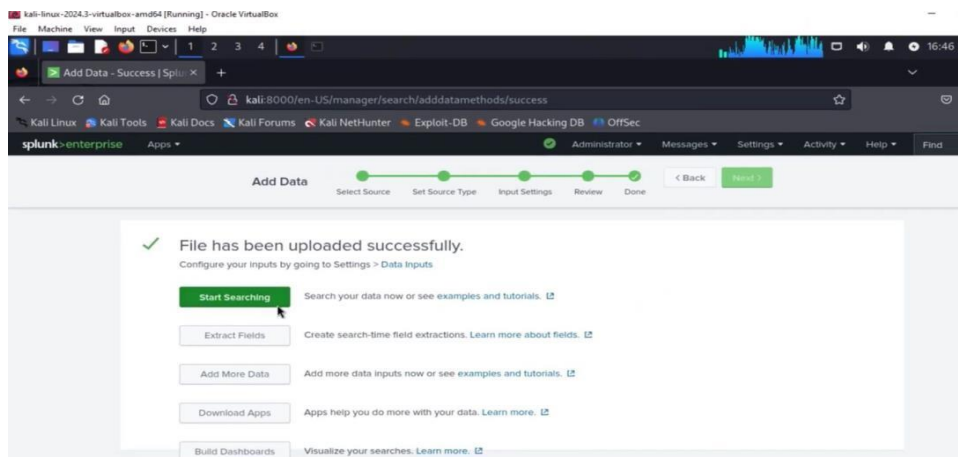
(kali@kali)-[/opt/splunk]
$ ls
bin          license-eula.txt  share
copyright.txt LICENSE.txt      splunk-9.4.0-6b4ebe426ca6-linux-amd64-manifest
etc          openssh.txt       swidtag
ftr          opt               var
include     quarantined_files
lib         README-splunk.txt

(kali@kali)-[/opt/splunk]
$ cd bin

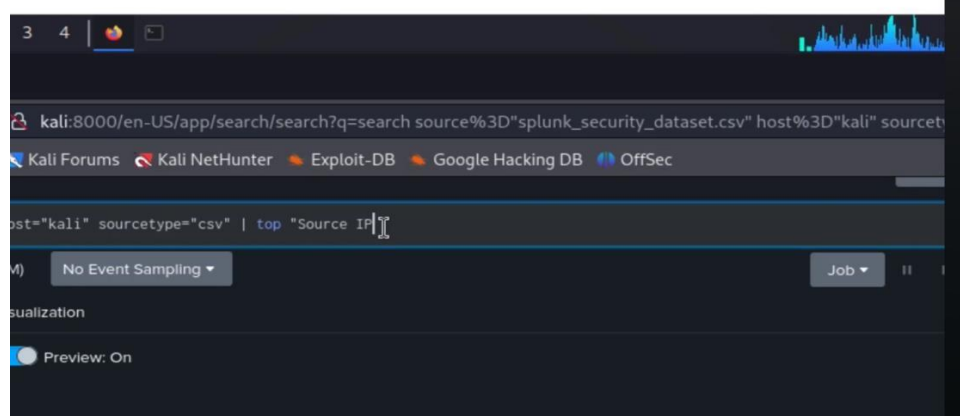
(kali@kali)-[/opt/splunk/bin]
$ sudo ./splunk start
[sudo] password for kali:
```

```
File Edit Search View Document Help
1 timestamp,Source IP,Destination IP,Event Type,Protocol,Event Details
2 15:04.1,192.168.1.15,10.0.0.3,Malware Download,HTTP,Malware Download detected on HTTP protocol
3 25:04.1,192.168.1.9,10.0.0.1,Malware Download,HTTP,Malware Download detected on HTTP protocol
4 25:04.1,192.168.1.18,10.0.0.4,Port Scan,SSH,Port Scan detected on SSH protocol
5 25:04.1,192.168.1.10,10.0.0.3,Phishing Email,TCP,Phishing Email detected on TCP protocol
6 55:04.1,192.168.1.3,10.0.0.2,Port Scan,TCP,Port Scan detected on TCP protocol
7 05:04.1,192.168.1.12,10.0.0.5,SQL Injection,TCP,SQL Injection detected on TCP protocol
8 45:04.1,192.168.1.16,10.0.0.4,SQL Injection,SMTP,SQL Injection detected on SMTP protocol
9 45:04.1,192.168.1.11,10.0.0.3,Malware Download,SSH,Malware Download detected on SSH protocol
10 25:04.1,192.168.1.18,10.0.0.5,Brute Force,TCP,Brute Force detected on TCP protocol
11 15:04.1,192.168.1.9,10.0.0.4,Port Scan,HTTP,Port Scan detected on HTTP protocol
12 25:04.1,192.168.1.16,10.0.0.1,Phishing Email,HTTP,Phishing Email detected on HTTP protocol
13 05:04.1,192.168.1.7,10.0.0.2,Port Scan,SSH,Port Scan detected on SSH protocol
14 35:04.1,192.168.1.17,10.0.0.5,Phishing Email,HTTP,Phishing Email detected on HTTP protocol
15 45:04.1,192.168.1.11,10.0.0.4,SQL Injection,SMTP,SQL Injection detected on SMTP protocol
16 15:04.1,192.168.1.15,10.0.0.3,SQL Injection,SMTP,SQL Injection detected on SMTP protocol
17 35:04.1,192.168.1.9,10.0.0.1,DDoS Attack,TCP,DDoS Attack detected on TCP protocol
18 35:04.1,192.168.1.14,10.0.0.1,SQL Injection,SMTP,SQL Injection detected on SMTP protocol
19 35:04.1,192.168.1.11,10.0.0.1,DDoS Attack,SMTP,DDoS Attack detected on SMTP protocol
20 45:04.1,192.168.1.17,10.0.0.1,Malware Download,SMTP,Malware Download detected on SMTP protocol
21 05:04.1,192.168.1.17,10.0.0.5,Port Scan,TCP,Port Scan detected on TCP protocol
22 45:04.1,192.168.1.5,10.0.0.1,Port Scan,SMTP,Port Scan detected on SMTP protocol
23 35:04.1,192.168.1.4,10.0.0.2,Brute Force,TCP,Brute Force detected on TCP protocol
```





le VirtualBox



Conclusion

This practical guides you through the steps to download, install, and configure Splunk on a Linux system. After completing these steps, you will have a functional Splunk instance running, ready for data collection and analysis.